

# Yulu - Case Study

by Suhel Mandal

## 1.Importing the libraries, Dataset and Basic Checks

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.stats import ttest_ind
from scipy.stats import chisquare
from scipy.stats import chi2
from scipy.stats import chi2_contingency
from statsmodels.stats.proportion import proportions_ztest
from statsmodels.graphics.gofplots import qqplot
from scipy.stats import f_oneway # Numeric Vs categorical for many categories
from scipy.stats import kruskal # Numeric Vs categorical for many categories
from scipy.stats import shapiro
from scipy.stats import levene
from scipy.stats import kstest

from IPython.display import HTML
import warnings
warnings.filterwarnings('ignore')

# style for all plots
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 5)
```

```
In [2]: from google.colab import files
uploaded = files.upload()
```

Choose Filesno files selected

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving bike\_sharing.csv to bike\_sharing.csv

```
In [3]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff;'>

yulu_df= pd.read_csv('/content/bike_sharing.csv')
yulu_df.head()
```

 Reading the Dataset and displaying the first 5 rows

Out[3]:

|   | datetime            | season | holiday | workingday | weather | temp | atemp  | humidity | windspeed | casual | registered | count |
|---|---------------------|--------|---------|------------|---------|------|--------|----------|-----------|--------|------------|-------|
| 0 | 2011-01-01 00:00:00 | 1      | 0       | 0          | 1       | 9.84 | 14.395 | 81       | 0.0       | 3      | 13         | 16    |
| 1 | 2011-01-01 01:00:00 | 1      | 0       | 0          | 1       | 9.02 | 13.635 | 80       | 0.0       | 8      | 32         | 40    |
| 2 | 2011-01-01 02:00:00 | 1      | 0       | 0          | 1       | 9.02 | 13.635 | 80       | 0.0       | 5      | 27         | 32    |
| 3 | 2011-01-01 03:00:00 | 1      | 0       | 0          | 1       | 9.84 | 14.395 | 75       | 0.0       | 3      | 10         | 13    |
| 4 | 2011-01-01 04:00:00 | 1      | 0       | 0          | 1       | 9.84 | 14.395 | 75       | 0.0       | 0      | 1          | 1     |

## 2.Data Types and Data Exploration

```
In [4]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff;'>
```

```
rows, cols = yulu_df.shape

print(f'The dataset contains {rows} rows and {cols} columns')
print('\n' + '*'*50)
print(f'\nColumn Names:\n{yulu_df.columns.to_list()}')
print('\n' + '*'*50)

display(HTML(f"""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

#### Dataset Shape and Column Overview

The dataset contains 10886 rows and 12 columns

\*\*\*\*\*

Column Names:

['datetime', 'season', 'holiday', 'workingday', 'weather', 'temp', 'atemp', 'humidity', 'windspeed', 'casual', 'registered', 'count']

\*\*\*\*\*

#### Insight:

- The dataset is moderately sized (10886 records), making it efficient for analysis.
- It contains 12 features, indicating a mix of variables that may require different preprocessing approaches.
- Understanding column names helps identify categorical and numerical features for further analysis.

```
In [5]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>

print(f'Showing the Data types of the Features:\n{yulu_df.dtypes}')

display(HTML("""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

#### Checking for Datatypes of the Features

Showing the Data types of the Features:

```
datetime      object
season        int64
holiday        int64
workingday     int64
weather        int64
temp          float64
atemp          float64
humidity       int64
windspeed     float64
casual         int64
registered     int64
count         int64
dtype: object
```

#### Insight:

- The 'datetime' column was stored as an object and has been converted to datetime format for time-based analysis.
- Numerical features appear correctly formatted for statistical analysis.

```
In [6]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>

unique_df = pd.DataFrame({'Column': yulu_df.columns, 'Unique Values': [yulu_df[col].nunique() for col in yulu_df.columns]})
display(unique_df)

display(HTML("""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

#### Unique Value Analysis

|    | Column     | Unique Values |
|----|------------|---------------|
| 0  | datetime   | 10886         |
| 1  | season     | 4             |
| 2  | holiday    | 2             |
| 3  | workingday | 2             |
| 4  | weather    | 4             |
| 5  | temp       | 49            |
| 6  | atemp      | 60            |
| 7  | humidity   | 89            |
| 8  | windspeed  | 28            |
| 9  | casual     | 309           |
| 10 | registered | 731           |
| 11 | count      | 822           |



#### Insight:

- Columns such as **season**, **holiday**, **workingday**, and **weather** contain a limited number of unique values, indicating categorical features.
- datetime** contains highly unique values since each record represents a specific timestamp.
- Continuous variables such as **temp**, **humidity**, and **windspeed** show higher variability and are suitable for numerical analysis.
- The dataset contains a mix of categorical and numerical features, making it appropriate for both exploratory analysis and predictive modeling.

```
In [7]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff;'>
null_values = yulu_df.isnull().sum()
display(null_values)

display(HTML("""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff;'>
```



#### Checking for Missing Values

|            |   |
|------------|---|
|            | 0 |
| datetime   | 0 |
| season     | 0 |
| holiday    | 0 |
| workingday | 0 |
| weather    | 0 |
| temp       | 0 |
| atemp      | 0 |
| humidity   | 0 |
| windspeed  | 0 |
| casual     | 0 |
| registered | 0 |
| count      | 0 |

dtype: int64



#### Insight:

- No missing values are present in the dataset, so no imputation is required.
- However, further data quality checks such as duplicates and outlier detection are necessary.

```
In [8]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff;'>
```

```
display(yulu_df.describe())

display(HTML("""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

#### Dataset Summary

|       | season       | holiday      | workingday   | weather      | temp         | atemp        | humidity     | windspeed    |
|-------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| count | 10886.000000 | 10886.000000 | 10886.000000 | 10886.000000 | 10886.000000 | 10886.000000 | 10886.000000 | 10886.000000 |
| mean  | 2.506614     | 0.028569     | 0.680875     | 1.418427     | 20.23086     | 23.655084    | 61.886460    | 12.79939     |
| std   | 1.116174     | 0.166599     | 0.466159     | 0.633839     | 7.79159      | 8.474601     | 19.245033    | 8.16453      |
| min   | 1.000000     | 0.000000     | 0.000000     | 1.000000     | 0.82000      | 0.760000     | 0.000000     | 0.00000      |
| 25%   | 2.000000     | 0.000000     | 0.000000     | 1.000000     | 13.94000     | 16.665000    | 47.000000    | 7.00150      |
| 50%   | 3.000000     | 0.000000     | 1.000000     | 1.000000     | 20.50000     | 24.240000    | 62.000000    | 12.99800     |
| 75%   | 4.000000     | 0.000000     | 1.000000     | 2.000000     | 26.24000     | 31.060000    | 77.000000    | 16.99790     |
| max   | 4.000000     | 1.000000     | 1.000000     | 4.000000     | 41.00000     | 45.455000    | 100.000000   | 56.99690     |

#### Insight:

- Check mean vs median for skewness
- Look for large std → high variability

```
In [9]: duplicate_count = yulu_df.duplicated().sum()

display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>

print(f'\nNumber of duplicate rows: {duplicate_count}')
```

```
if duplicate_count == 0:
    display(HTML("""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
else:
    display(HTML(f""<div style='background-color:#fff3cd; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

#### Checking for Duplicates

Number of duplicate rows: 0

#### Insight:

- No duplicate rows are present in the dataset, indicating good data quality.
- No duplicate treatment or row removal is required before analysis.

## 3.Convert Date Feature

```
In [10]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>

# Convert to datetime
yulu_df['datetime'] = pd.to_datetime(yulu_df['datetime'])

# Verify conversion
display(yulu_df.dtypes)

# Extracting useful features
yulu_df['year'] = yulu_df['datetime'].dt.year
yulu_df['month'] = yulu_df['datetime'].dt.month
yulu_df['day'] = yulu_df['datetime'].dt.day
yulu_df['hour'] = yulu_df['datetime'].dt.hour

display(HTML("""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

#### Converting 'datetime' Column to Datetime Format

0

|                   |                |
|-------------------|----------------|
| <b>datetime</b>   | datetime64[ns] |
| <b>season</b>     | int64          |
| <b>holiday</b>    | int64          |
| <b>workingday</b> | int64          |
| <b>weather</b>    | int64          |
| <b>temp</b>       | float64        |
| <b>atemp</b>      | float64        |
| <b>humidity</b>   | int64          |
| <b>windspeed</b>  | float64        |
| <b>casual</b>     | int64          |
| <b>registered</b> | int64          |
| <b>count</b>      | int64          |

dtype: object

 **Insight:**

- The 'datetime' column has been successfully converted to datetime format.
- This enables time-based analysis such as trends by year, month, or hour.
- New features like year, month, day, and hour have been extracted to support deeper analysis.

## 4.Non-Graphical Analysis

In [11]:

```
# Convert datetime and extract just the date
yulu_df['date'] = yulu_df['datetime'].dt.date

# Now count unique days
daily_df = yulu_df.groupby('date').first().reset_index()

# Now these counts reflect actual days, not hours
print(daily_df['holiday'].value_counts())
print(daily_df['workingday'].value_counts())
```

holiday  
0 443  
1 13  
Name: count, dtype: int64  
workingday  
1 311  
0 145  
Name: count, dtype: int64

In [12]:

```
display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff;'>

yulu_df['date'] = pd.to_datetime(yulu_df['datetime']).dt.date
daily_df = yulu_df.groupby('date').first().reset_index()

holiday_counts = daily_df['holiday'].value_counts()
display(holiday_counts)

total_days = holiday_counts.sum()
holiday_days = holiday_counts.get(1, 0)
non_holiday_days = holiday_counts.get(0, 0)

display(HTML(f""""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff;'>


```

 **Distribution of Holiday Feature**

|         | count |
|---------|-------|
| holiday |       |
| 0       | 443   |
| 1       | 13    |

dtype: int64

 **Insight:**

- The dataset spans **456 days** in total (approx. 15 months of hourly data).
- Out of these, only **13 days (2.9%)** are holidays, while **443 days (97.1%)** are non-holidays.
- This is a **severe class imbalance** — holidays are genuinely rare in real-world data, which is expected, but it limits our ability to draw strong statistical conclusions about holiday behaviour.
- With only 13 holiday days, any analysis involving holidays should be **interpreted with caution** as the sample may not be representative enough.
- For the hypothesis tests ahead, we use **workingday** (which is more balanced at 68.2% working vs 31.8% non-working) as a more reliable predictor of demand.

```
In [13]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff;'>

wd_counts = daily_df['workingday'].value_counts()
display(wd_counts)

total_days = wd_counts.sum()
working_days = wd_counts.get(1, 0)
non_working_days = wd_counts.get(0, 0)
working_pct = round(working_days / total_days * 100, 1)
non_working_pct = round(non_working_days / total_days * 100, 1)

# Average rentals on working vs non-working days
avg_working = round(yulu_df[yulu_df['workingday'] == 1]['count'].mean(), 2)
avg_non_working = round(yulu_df[yulu_df['workingday'] == 0]['count'].mean(), 2)
higher = "working days" if avg_working > avg_non_working else "non-working days"

#Displaying the Data

summary = pd.DataFrame({'Category': ['Working Day', 'Non-Working Day'],'Day Count': [working_days, non_working_days]})
display(summary.style.set_caption("Working Day Summary").highlight_max(subset=['Day Count', 'Avg Hourly Rentals']))
display(HTML(f""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff;'>
```

 **Checking the Working Day Count**

|            | count |
|------------|-------|
| workingday |       |
| 1          | 311   |
| 0          | 145   |

dtype: int64

| Working Day Summary |                 |           |                |                    |
|---------------------|-----------------|-----------|----------------|--------------------|
|                     | Category        | Day Count | Percentage (%) | Avg Hourly Rentals |
| 0                   | Working Day     | 311       | 68.2%          | 193.01             |
| 1                   | Non-Working Day | 145       | 31.8%          | 188.51             |

### Insight:

- Out of **456 total days**, **311 days (68.2%)** are working days and **145 days (31.8%)** are non-working days (weekends/holidays).
- The dataset is **moderately imbalanced** but usable — working days outnumber non-working days roughly **2:1**, which reflects a real-world calendar.
- Average hourly rentals on **working days: 193.01** vs **non-working days: 188.51** — demand is higher on **working days**.
- This early difference in means suggests working day status **may significantly affect demand** — which we will formally verify using a **2-sample T-test** in the hypothesis testing section.

```
In [14]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>

daily_df['day_type'] = 'Working Day'
daily_df.loc[daily_df['workingday'] == 0, 'day_type'] = 'Weekend'
daily_df.loc[daily_df['holiday'] == 1, 'day_type'] = 'Holiday'

day_type_counts = daily_df['day_type'].value_counts()
display(day_type_counts)

total = day_type_counts.sum()
wd = day_type_counts.get('Working Day', 0)
we = day_type_counts.get('Weekend', 0)
hol = day_type_counts.get('Holiday', 0)

avg_working = round(yulu_df[yulu_df['workingday'] == 1]['count'].mean(), 2)
avg_weekend = round(yulu_df[(yulu_df['workingday'] == 0) & (yulu_df['holiday'] == 0)]['count'].mean(), 2)
avg_holiday = round(yulu_df[yulu_df['holiday'] == 1]['count'].mean(), 2)

percent_wd = round(wd / total * 100, 1)
percent_we = round(we / total * 100, 1)
percent_hol = round(hol / total * 100, 1)

summary_day_types = pd.DataFrame({'Category': ['Working Day', 'Weekend', 'Holiday'], 'Day Count': [wd, we, hol],
                                  'Percentage (%)': [percent_wd, percent_we, percent_hol],
                                  'Avg Hourly Rentals': [avg_working, avg_weekend, avg_holiday]})

display(summary_day_types.style.set_caption("Day Type Summary").highlight_max(subset=['Day Count', 'Avg Hourly Rentals']))
display(HTML(f""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

### Distribution of Day Types (Working Day, Weekend, Holiday)

| count       |     |
|-------------|-----|
| day_type    |     |
| Working Day | 311 |
| Weekend     | 132 |
| Holiday     | 13  |

dtype: int64

| Day Type Summary |             |           |                |                    |
|------------------|-------------|-----------|----------------|--------------------|
|                  | Category    | Day Count | Percentage (%) | Avg Hourly Rentals |
| 0                | Working Day | 311       | 68.2%          | 193.01             |
| 1                | Weekend     | 132       | 28.9%          | 188.77             |
| 2                | Holiday     | 13        | 2.9%           | 185.88             |

### Insight:

- Out of **456 total days: 311 (68.2%)** are Working Days, **132 (28.9%)** are Weekends, and **13 (2.9%)** are Holidays.
- Working days dominate the dataset (~2:1 ratio over weekends), which reflects a normal calendar structure.
- Holidays are extremely rare at only 13 days — making it statistically difficult to draw strong conclusions about holiday-specific demand.
- For hypothesis testing, **workingday** (working vs non-working) is a more reliable grouping than holiday alone, since it has a more balanced split.

```
In [15]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>

season_counts = daily_df['season'].map({1:'Spring', 2:'Summer', 3:'Fall', 4:'Winter'}).value_counts()
display(season_counts)
```

```
total_days = season_counts.sum()
max_season = season_counts.idxmax()
min_season = season_counts.idxmin()

display(HTML(f"""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

 **Season Count (1: Spring, 2: Summer, 3: Fall, 4: Winter)**

|        | count |
|--------|-------|
| season |       |
| Spring | 114   |
| Summer | 114   |
| Fall   | 114   |
| Winter | 114   |

dtype: int64

 **Insight:**

- The dataset spans **456 days** and is **perfectly balanced** across all four seasons with exactly **114 days each**.
- This perfect balance is likely due to how the dataset was constructed — it ensures **no seasonal bias** in our analysis.
- Since all seasons have equal representation, any differences in rental demand across seasons can be attributed to **actual seasonal effects** and not data imbalance.
- This makes the upcoming **ANOVA test across seasons** more reliable and fair.

```
In [16]: display(HTML(f"""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>

seasons_mapping = {1: 'Spring', 2: 'Summer', 3: 'Fall', 4: 'Winter'}
weather_mapping = {1: 'Clear', 2: 'Mist/Cloudy', 3: 'Light Rain/Snow', 4: 'Heavy Rain/Snow/Fog'}

season_weather = (yulu_df.assign(mapped_season = yulu_df['season'].map(seasons_mapping),mapped_weather = yulu_df['weather'].map(weather_mapping))
pivot = season_weather.pivot(index='mapped_season',columns='mapped_weather',values='count').fillna(0).astype(int)
display(pivot.style.set_caption("Total Rentals by Season and Weather").background_gradient(cmap='Blues', axis=1))

# Dynamic insights
clear_dominant = pivot['Clear'].idxmax()
mist_highest = pivot['Mist/Cloudy'].idxmax()
rain_lowest = pivot['Light Rain/Snow'].idxmin()
best_season = pivot.sum(axis=1).idxmax()
worst_season = pivot.sum(axis=1).idxmin()

display(HTML(f"""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

 **Season wise Weather - Total Rentals**

| Total Rentals by Season and Weather |         |                     |                 |             |
|-------------------------------------|---------|---------------------|-----------------|-------------|
| mapped_weather                      | Clear   | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
| mapped_season                       |         |                     |                 |             |
| Fall                                | 470,116 | 0                   | 31,160          | 139,386     |
| Spring                              | 223,009 | 164                 | 12,919          | 76,406      |
| Summer                              | 426,350 | 0                   | 27,755          | 134,177     |
| Winter                              | 356,588 | 0                   | 30,255          | 157,191     |

 **Insight:**

- ☀️ **Clear weather drives the highest rentals across all seasons** — most prominent in **Fall**.
- ☁️ **Mist/Cloudy rentals are highest in Winter** — even in poor weather, this season maintains decent demand.
- 🌧️ **Light Rain/Snow rentals are lowest in Spring** — riders in this season are most deterred by bad weather.
- 🏆 **Fall has the highest total rentals** overall across all weather conditions.
- 📉 **Spring has the lowest total rentals** overall — likely due to less favorable weather and temperature.
- ⚠️ **Heavy Rain/Snow** has negligible rentals — only 1 recorded hour in the dataset, too rare to be meaningful.

```
In [17]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>

temp_summary = (yulu_df.assign(mapped_season = yulu_df['season'].map(seasons_mapping),mapped_weather = yulu_df['weather'].map(weather_mapping))

# Pivot for temp
temp_pivot = temp_summary.pivot(index='mapped_season',columns='mapped_weather',values='temp').fillna(0).round(2)

# Pivot for atemp
atemp_pivot = temp_summary.pivot(index='mapped_season',columns='mapped_weather',values='atemp').fillna(0).round(2)

print("📌 Average Actual Temperature (°C) by Season and Weather:")
display(temp_pivot.style.set_caption("Avg Temperature (°C)").background_gradient(cmap='Reds', axis=None).format(formatter="{:.2f}°C"))

print("📌 Average Feels-Like Temperature (°C) by Season and Weather:")
display(atemp_pivot.style.set_caption("Avg Feels-Like Temperature (°C)").background_gradient(cmap='Oranges', axis=None).format(formatter="{:.2f}°C"))

# Dynamic insights
hottest_season = temp_pivot.mean(axis=1).idxmax()
coldest_season = temp_pivot.mean(axis=1).idxmin()
corr = round(yulu_df['temp'].corr(yulu_df['atemp']), 4)

display(HTML(f"""" <div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>

```

 **Average Temperature vs Feels-like Temperature by Season and Weather**

📌 Average Actual Temperature (°C) by Season and Weather:  
Avg Temperature (°C)

| mapped_weather | Clear   | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
|----------------|---------|---------------------|-----------------|-------------|
| mapped_season  |         |                     |                 |             |
| Fall           | 29.23°C | 0.00°C              | 26.79°C         | 28.05°C     |
| Spring         | 12.54°C | 8.20°C              | 12.15°C         | 12.63°C     |
| Summer         | 23.18°C | 0.00°C              | 21.00°C         | 22.49°C     |
| Winter         | 16.24°C | 0.00°C              | 18.63°C         | 16.97°C     |

📌 Average Feels-Like Temperature (°C) by Season and Weather:  
Avg Feels-Like Temperature (°C)

| mapped_weather | Clear   | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
|----------------|---------|---------------------|-----------------|-------------|
| mapped_season  |         |                     |                 |             |
| Fall           | 33.04°C | 0.00°C              | 29.98°C         | 31.77°C     |
| Spring         | 15.14°C | 11.36°C             | 14.94°C         | 15.55°C     |
| Summer         | 27.03°C | 0.00°C              | 24.50°C         | 26.35°C     |
| Winter         | 19.67°C | 0.00°C              | 21.95°C         | 20.36°C     |

 **Insight:**

- 🔧 **temp** and **atemp** follow almost identical patterns across all season-weather combinations — their correlation is **0.9849**, confirming near-perfect alignment.
- ☀️ **Fall** records the highest average temperatures across all weather conditions — making it the most comfortable season for riding.
- ❄️ **Spring** records the lowest average temperatures — adverse weather in this season compounds the cold, further reducing rider comfort.
- ☁️ Rainy and snowy conditions are consistently associated with lower temperatures across all seasons — bad weather and cold tend to occur together.
- 📈 Since **temp** and **atemp** have a correlation of **0.9849**, using both in a regression model would cause **multicollinearity** — only one should be kept for predictive modelling.

```
In [18]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>

humidity_pivot = yulu_df.assign(mapped_season=yulu_df['season'].map(seasons_mapping), mapped_weather=yulu_df['weather'].map(weather_mapping))

display(humidity_pivot.style.set_caption("Average Humidity (%) by Season and Weather").background_gradient(cmap='Blues', axis=None).format(formatter="{:.2f}%"))

most_humid_season, least_humid_season = humidity_pivot.mean(axis=1).idxmax(), humidity_pivot.mean(axis=1).idxmin()
most_humid_weather, least_humid_weather = humidity_pivot.mean(axis=0).idxmax(), humidity_pivot.mean(axis=0).idxmin()
corr_humidity_count = round(yulu_df['humidity'].corr(yulu_df['count']), 4)

display(HTML(f"""" <div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>

```

### Average Humidity by Season and Weather

Average Humidity (%) by Season and Weather

| mapped_weather | Clear  | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
|----------------|--------|---------------------|-----------------|-------------|
| mapped_season  |        |                     |                 |             |
| Fall           | 60.59% | 0.00%               | 82.09%          | 69.50%      |
| Spring         | 50.12% | 86.00%              | 76.47%          | 65.51%      |
| Summer         | 54.93% | 0.00%               | 81.15%          | 69.50%      |
| Winter         | 61.04% | 0.00%               | 85.44%          | 71.63%      |

#### Insight:

-  **Spring** is the most humid season while **Summer** is the driest — drier conditions encourage more riding.
-  **Light Rain/Snow** has the highest humidity while **Heavy Rain/Snow/Fog** has the lowest — adverse weather brings higher moisture.
-  Clear weather consistently shows the lowest humidity — most comfortable riding conditions.
-  Humidity-rental correlation = **-0.3174** — higher humidity associated with **lower** demand.
-  Yulu should factor humidity forecasts into daily fleet deployment planning.

```
In [19]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>
windspeed_pivot = yulu_df.assign(mapped_season=yulu_df['season'].map(seasons_mapping), mapped_weather=yulu_
display(windspeed_pivot.style.set_caption("Average Windspeed by Season and Weather").background_gradient(cmap='Blues', axis=None))

windiest_season, calmest_season = windspeed_pivot.mean(axis=1).idxmax(), windspeed_pivot.mean(axis=1).idxmin()
windiest_weather, calmest_weather = windspeed_pivot.mean(axis=0).idxmax(), windspeed_pivot.mean(axis=0).idxmin()
corr_wind_count = round(yulu_df['windspeed'].corr(yulu_df['count']), 4)

display(HTML(f""""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

### Average Windspeed by Season and Weather

Average Windspeed by Season and Weather

| mapped_weather | Clear | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
|----------------|-------|---------------------|-----------------|-------------|
| mapped_season  |       |                     |                 |             |
| Fall           | 11.24 | -                   | 14.40           | 11.41       |
| Spring         | 15.74 | 6.00                | 12.37           | 12.62       |
| Summer         | 13.28 | -                   | 15.78           | 12.97       |
| Winter         | 11.41 | -                   | 13.67           | 11.68       |

#### Insight:

-  **Summer** is the windiest season while **Spring** has the calmest conditions — calmer weather encourages more riding.
-  **Light Rain/Snow** has the highest average windspeed — strong winds accompany adverse weather, discouraging riders.
-  **Heavy Rain/Snow/Fog** is the calmest condition — low wind + clear skies = ideal riding conditions.
-  Windspeed-rental correlation = **0.1014** — windspeed has a **weak positive** effect on demand.
-  Yulu should monitor wind forecasts — high windspeed days may need reduced fleet deployment in exposed zones.

```
In [20]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>
user_summary = yulu_df.assign(mapped_season=yulu_df['season'].map(seasons_mapping), mapped_weather=yulu_df['weather'].map(weather_mapping))

casual_pivot = user_summary.pivot(index='mapped_season', columns='mapped_weather', values='casual').fillna(0)
registered_pivot = user_summary.pivot(index='mapped_season', columns='mapped_weather', values='registered').fillna(0)

print("🚗 Average Casual Users by Season and Weather:")
display(casual_pivot.style.set_caption("Avg Casual Users").background_gradient(cmap='Oranges', axis=None))

print("🚗 Average Registered Users by Season and Weather:")
display(registered_pivot.style.set_caption("Avg Registered Users").background_gradient(cmap='Blues', axis=None))

top_casual_season, top_registered_season = casual_pivot.mean(axis=1).idxmax(), registered_pivot.mean(axis=1).idxmax()
```

```

top_casual_weather, top_reg_weather = casual_pivot.mean(axis=0).idxmax(), registered_pivot.mean(axis=0).idxmax()
avg_casual, avg_registered = round(yulu_df['casual'].mean(), 2), round(yulu_df['registered'].mean(), 2)
ratio = round(avg_registered / avg_casual, 1)
casual_drop = round(((casual_pivot['Clear'].mean() - casual_pivot['Light Rain/Snow'].mean()) / casual_pivot['Clear'].mean()) * 100, 1)
registered_drop = round(((registered_pivot['Clear'].mean() - registered_pivot['Light Rain/Snow'].mean()) / registered_pivot['Clear'].mean()) * 100, 1)

display(HTML(f"""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>

```

## 🚲 Average Casual and Registered Users by Season and Weather

📌 Average Casual Users by Season and Weather:

Avg Casual Users

| mapped_weather | Clear | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
|----------------|-------|---------------------|-----------------|-------------|
| mapped_season  |       |                     |                 |             |
| Fall           | 54.97 | -                   | 32.92           | 49.78       |
| Spring         | 17.65 | 6.00                | 3.53            | 13.72       |
| Summer         | 53.13 | -                   | 20.59           | 41.49       |
| Winter         | 33.53 | -                   | 13.67           | 22.29       |

📌 Average Registered Users by Season and Weather:

Avg Registered Users

| mapped_weather | Clear  | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
|----------------|--------|---------------------|-----------------|-------------|
| mapped_season  |        |                     |                 |             |
| Fall           | 188.61 | -                   | 123.66          | 180.99      |
| Spring         | 109.13 | 158.00              | 57.70           | 93.14       |
| Summer         | 183.60 | -                   | 103.32          | 148.02      |
| Winter         | 175.98 | -                   | 120.80          | 172.49      |

## 📊 Insight:

- 👤 Registered users (**155.55**) outnumber casual users (**36.02**) by **4.3:1** — service primarily driven by regular commuters.
- ☀️ Both user types peak under **Clear** conditions — clear weather is the strongest demand driver for both groups.
- ☁️ From Clear to Light Rain/Snow, casual users drop by **55.6%** vs registered dropping only **38.3%** — casual riders are significantly more weather-sensitive.
- 🏆 **Fall** has peak casual usage while **Fall** has peak registered usage — different seasonal motivations between groups.
- 📈 Yulu should target **casual-to-registered conversion** during peak seasons — would significantly stabilise revenue during bad weather.

```

In [21]: display(HTML(f"""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>

# Segregating the time of day
yulu_df['Day_Time'] = yulu_df['hour'].apply(lambda x:
    'Early Morning' if 5 <= x < 7
    else 'Morning'   if 7 <= x < 12
    else 'Afternoon' if 12 <= x < 18
    else 'Evening'   if 18 <= x < 21
    else 'Night & Late Night')

day_time_count = yulu_df.groupby('Day_Time')['count'].sum()
day_time_mean  = yulu_df.groupby('Day_Time')['count'].mean().round(2)

display(day_time_count.to_frame().style.set_caption("Total Rentals by Time of Day").background_gradient(cmap=plasma))
display(day_time_mean.to_frame().style.set_caption("Average Rentals by Time of Day").background_gradient(cmap=plasma))

# Dynamic insights
day_time_count_max = day_time_count.idxmax()
day_time_count_min = day_time_count.idxmin()
day_time_mean_max  = day_time_mean.idxmax()
day_time_mean_min  = day_time_mean.idxmin()

pct_afternoon = round(day_time_count['Afternoon'] / day_time_count.sum() * 100, 1)
pct_night     = round(day_time_count['Night & Late Night'] / day_time_count.sum() * 100, 1)

display(HTML(f"""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>

```

## Total Rentals Based On The Time of Day

Total Rentals by Time of Day

|                    | count   |
|--------------------|---------|
| Day_Time           |         |
| Afternoon          | 819,512 |
| Early Morning      | 43,633  |
| Evening            | 444,443 |
| Morning            | 538,462 |
| Night & Late Night | 239,426 |

Average Rentals by Time of Day

|                    | count  |
|--------------------|--------|
| Day_Time           |        |
| Afternoon          | 299.53 |
| Early Morning      | 48.11  |
| Evening            | 324.89 |
| Morning            | 236.69 |
| Night & Late Night | 66.51  |

### Insight:

- 🏆 **Afternoon** has the highest total rentals, accounting for **39.3%** of all rides — this is the peak demand window for Yulu.
- 😴 **Early Morning** has the lowest total rentals at just **11.5%** of all rides — very few users ride during late hours.
- 📈 **Evening** also has the highest average rentals per hour — confirming it is consistently busy, not just occasionally.
- 🌅 **Early Morning** has low demand, suggesting Yulu cycles are rarely used before 7am — late starters dominate the user base.
- 🚲 **Business Recommendation:** Yulu should ensure maximum cycle availability during Afternoon hours, and consider rebalancing cycles from Early Morning zones to high-demand areas proactively.

## 5.Univariate Analysis

```
In [22]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>

continuous_cols = ['temp', 'atemp', 'humidity', 'windspeed', 'count', 'casual', 'registered']

fig, axes = plt.subplots(len(continuous_cols),2,figsize=(14,4*len(continuous_cols)))

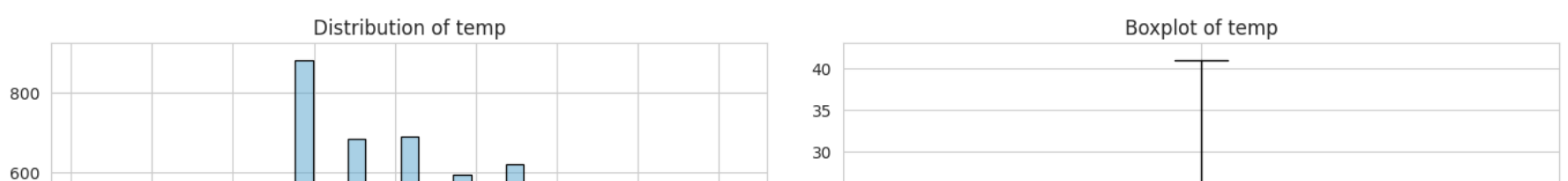
colorp = sns.color_palette("icefire")

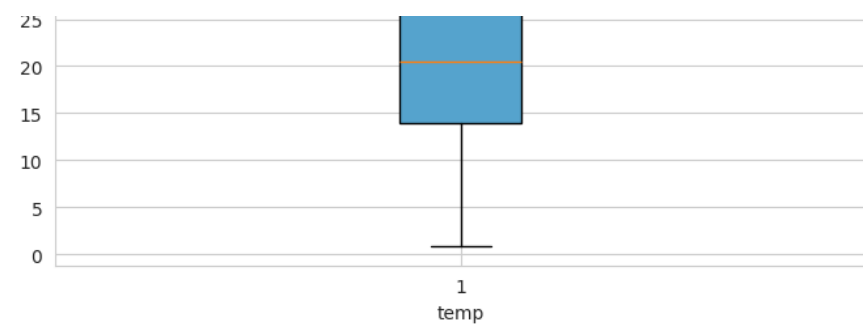
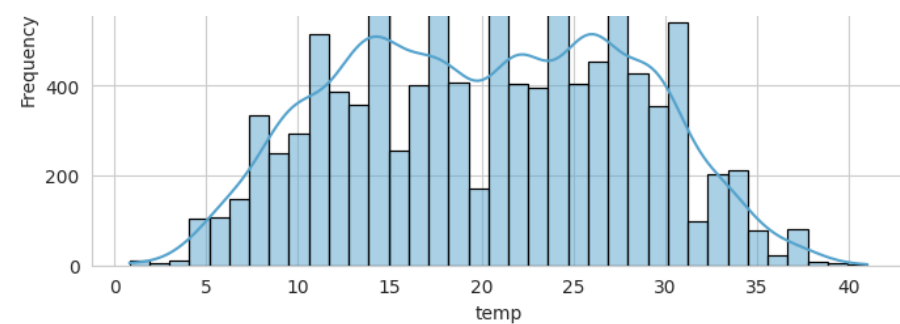
for i,col in enumerate(continuous_cols):
    sns.histplot(yulu_df[col], ax = axes[i,0], color = colorp[i % len(colorp)],kde = True, edgecolor = 'black')
    axes[i,0].set_title(f'Distribution of {col}', fontsize = 12)
    axes[i,0].set_xlabel(col)
    axes[i,0].set_ylabel('Frequency')

    axes[i,1].boxplot(yulu_df[col], vert=True, patch_artist=True,
                      boxprops=dict(facecolor=colorp[i % len(colorp)]))
    axes[i,1].set_title(f'Boxplot of {col}', fontsize=12)
    axes[i,1].set_xlabel(col)

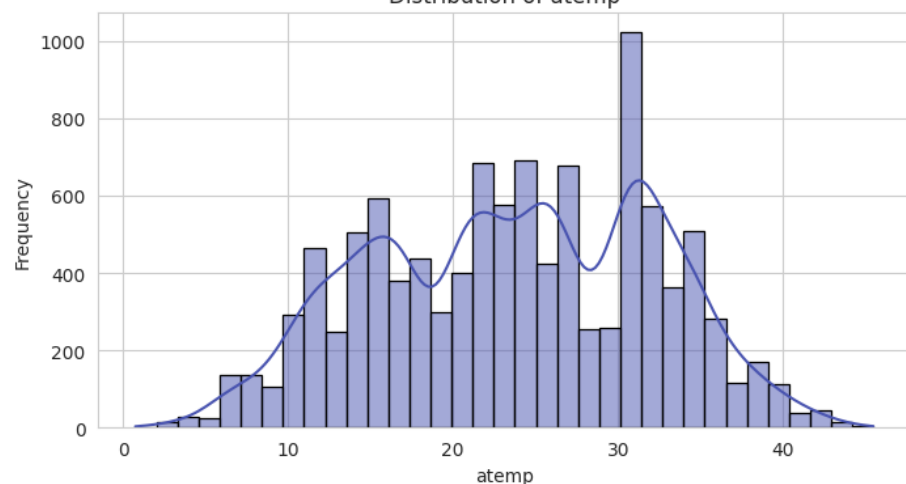
plt.tight_layout()
plt.show()
display(HTML("""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

### Univariate Analysis – Continuous Variables

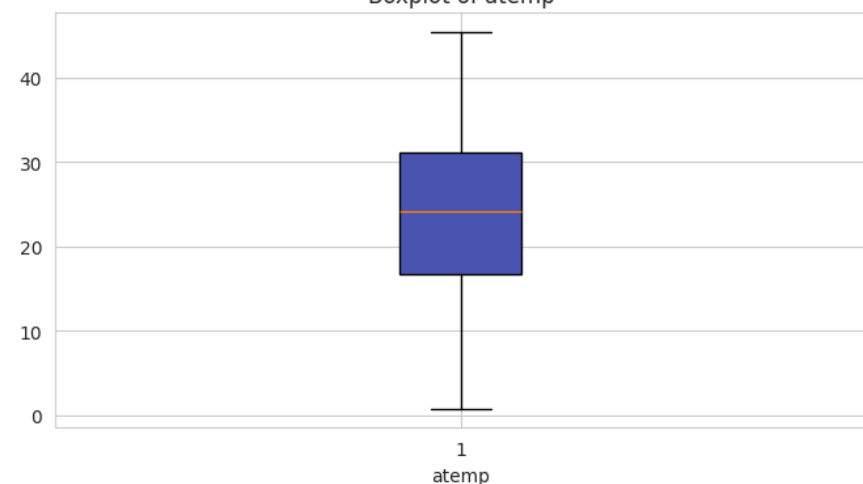




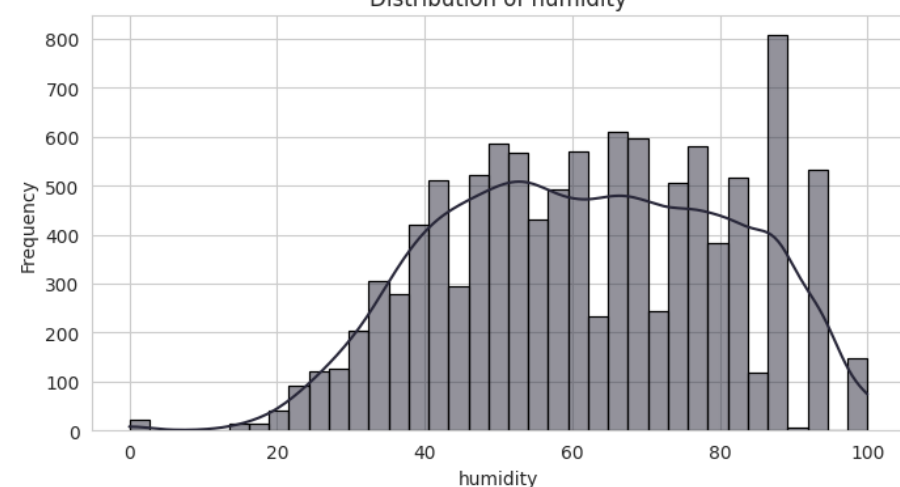
Distribution of atemp



Boxplot of atemp



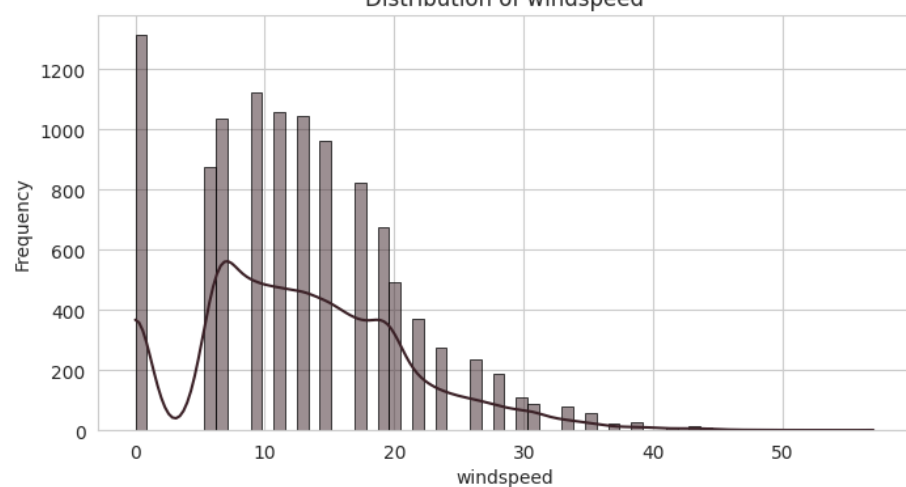
Distribution of humidity



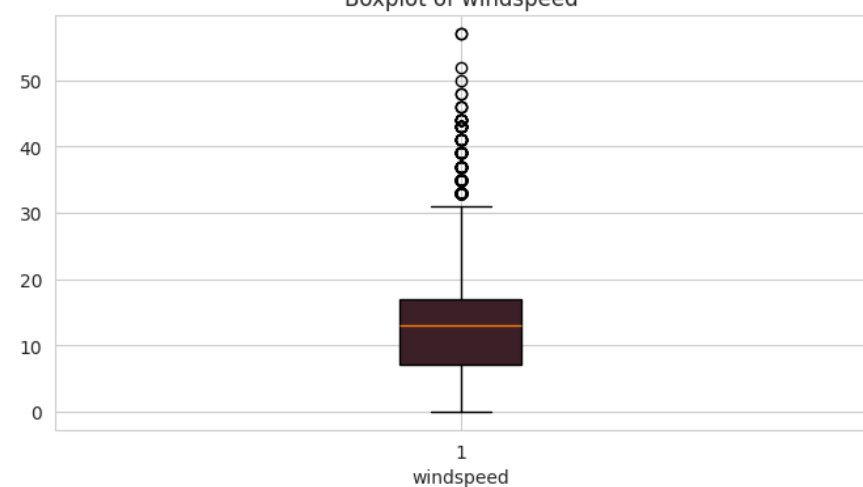
Boxplot of humidity



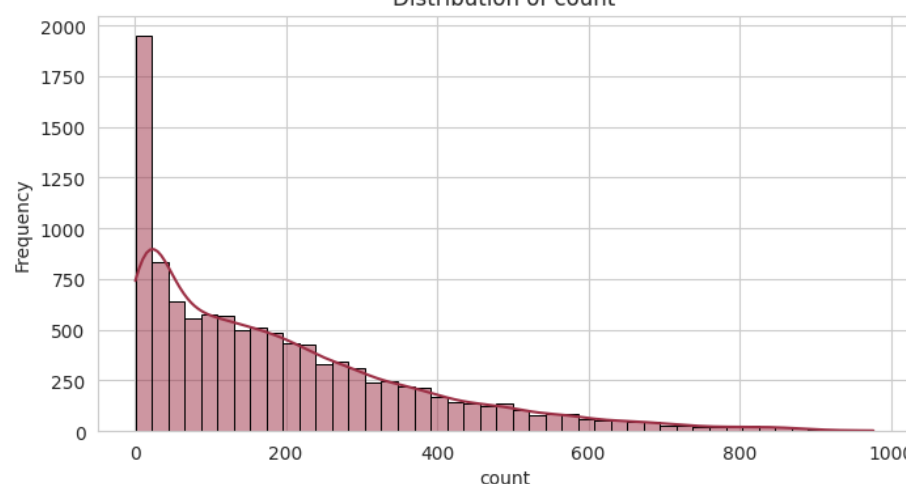
Distribution of windspeed



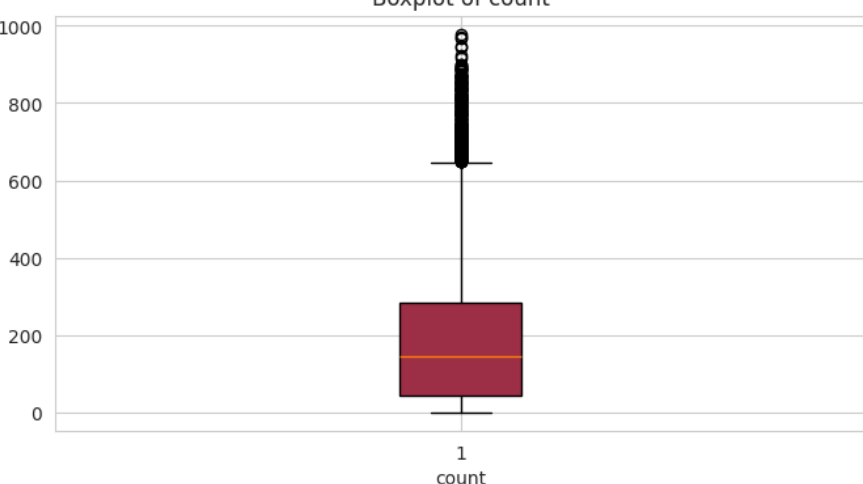
Boxplot of windspeed



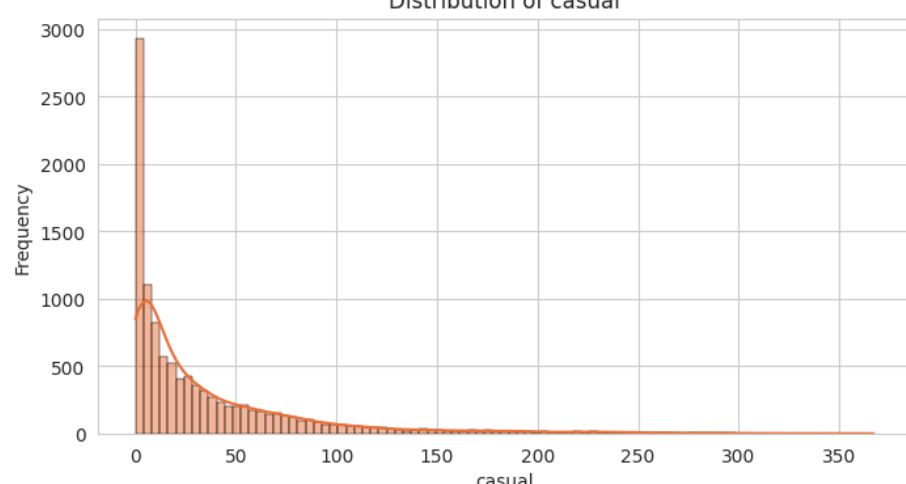
Distribution of count



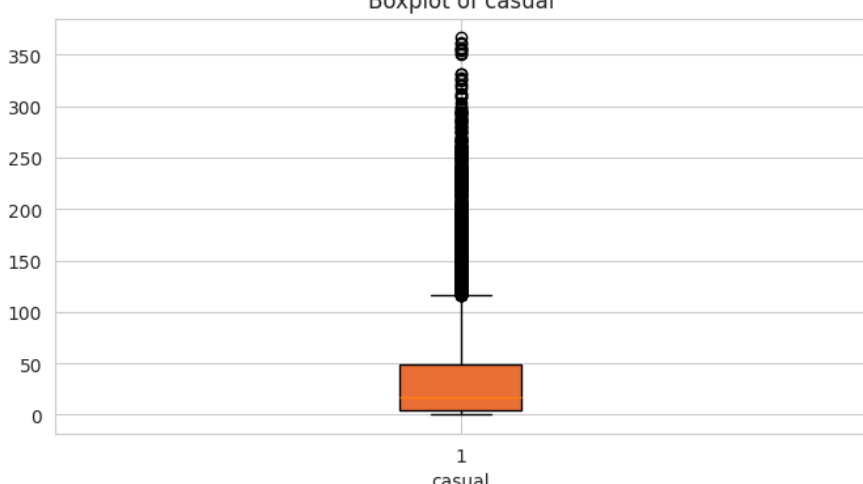
Boxplot of count



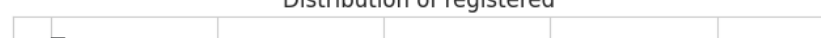
Distribution of casual



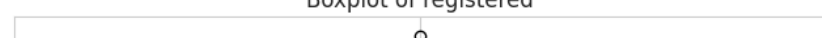
Boxplot of casual

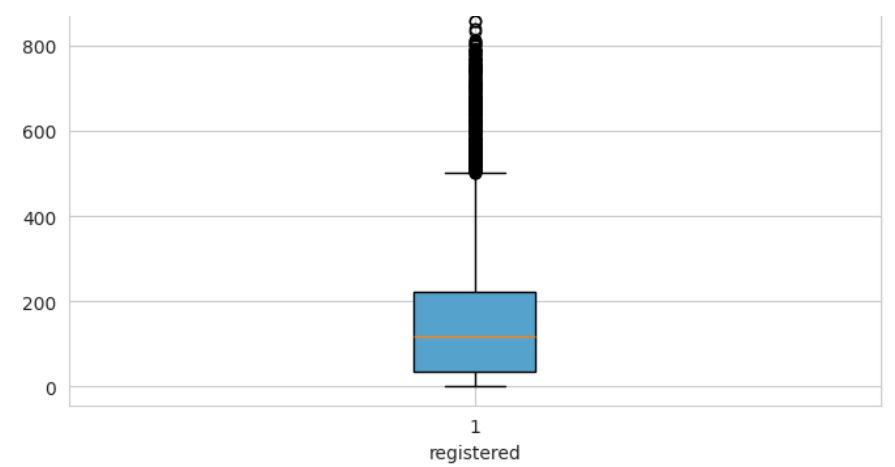
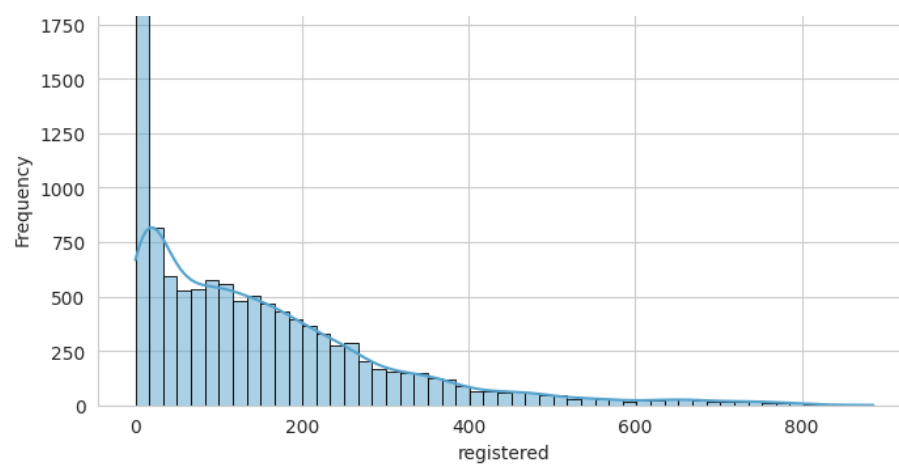


Distribution of registered



Boxplot of registered





#### Insight:

- **count, casual, registered:** All right-skewed — most rentals are low-volume hours, with occasional high-demand peaks.
- **temp & atemp:** Roughly uniform/bimodal — data spans a wide range of temperatures.
- **humidity:** Slightly left-skewed, most readings are moderate-to-high humidity.
- **windspeed:** Right-skewed with many zero values — likely recorded as 0 when not measured.
- Boxplots confirm outliers exist in casual, registered, count, and windspeed.

```
In [23]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid
plt.figure(figsize=(16, 5))

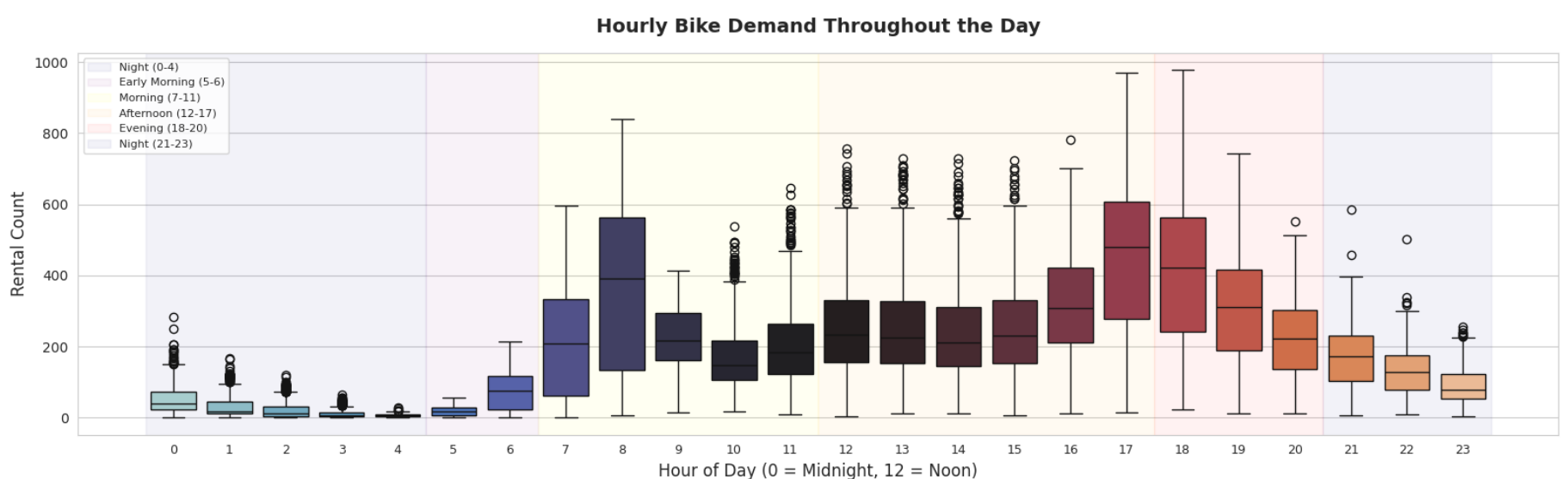
sns.boxplot(x='hour',y='count',data=yulu_df,palette='icefire')

plt.title('Hourly Bike Demand Throughout the Day', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Hour of Day (0 = Midnight, 12 = Noon)', fontsize=12)
plt.ylabel('Rental Count', fontsize=12)
plt.xticks(range(0, 24), fontsize=9)

# Add vertical lines to separate time periods
plt.axvspan(-0.5, 4.5, alpha=0.05, color='navy', label='Night (0-4)')
plt.axvspan(4.5, 6.5, alpha=0.05, color='purple', label='Early Morning (5-6)')
plt.axvspan(6.5, 11.5, alpha=0.05, color='yellow', label='Morning (7-11)')
plt.axvspan(11.5, 17.5, alpha=0.05, color='orange', label='Afternoon (12-17)')
plt.axvspan(17.5, 20.5, alpha=0.05, color='red', label='Evening (18-20)')
plt.axvspan(20.5, 23.5, alpha=0.05, color='navy', label='Night (21-23)')

plt.legend(fontsize=8, loc='upper left')
plt.tight_layout()
plt.show()
```

#### Hourly Basis Count throuout the day



```
In [24]: # — Existing Dynamic values —————
skewness = yulu_df[continuous_cols].skew().round(3)
outlier_counts = {}
for col in continuous_cols:
    Q1 = yulu_df[col].quantile(0.25)
    Q3 = yulu_df[col].quantile(0.75)
    IQR = Q3 - Q1
    outliers = ((yulu_df[col] < Q1 - 1.5*IQR) | (yulu_df[col] > Q3 + 1.5*IQR)).sum()
    outlier_counts[col] = outliers

most_skewed = skewness.abs().idxmax()
most_outliers = max(outlier_counts, key=outlier_counts.get)
zero_wind = (yulu_df['windspeed'] == 0).sum()
zero_wind_pct = round(zero_wind / len(yulu_df) * 100, 1)
avg_count = round(yulu_df['count'].mean(), 1)
```

```

max_count      = yulu_df['count'].max()
corr_temp_atemp = round(yulu_df['temp'].corr(yulu_df['atemp']), 4)

# — Hourly Dynamic values —
hourly_avg      = yulu_df.groupby('hour')['count'].mean()
peak_hour       = hourly_avg.idxmax()
lowest_hour     = hourly_avg.idxmin()
peak_hour_avg   = round(hourly_avg.max(), 1)
lowest_hour_avg = round(hourly_avg.min(), 1)
demand_gap      = round(peak_hour_avg - lowest_hour_avg, 1)
top3_hours      = hourly_avg.nlargest(3).index.tolist()
bottom3_hours   = hourly_avg.nsmallest(3).index.tolist()

def get_period(h):
    if 5 <= h < 7: return 'Early Morning'
    if 7 <= h < 12: return 'Morning'
    if 12 <= h < 18: return 'Afternoon'
    if 18 <= h < 21: return 'Evening'
    return 'Night & Late Night'

peak_period     = get_period(peak_hour)
lowest_period   = get_period(lowest_hour)

display(HTML(f"""<div style='background-color:#f0f8ff; padding:15px; border-radius:8px; border-left:5px solid

```

### Insight — Continuous Variables & Hourly Demand:

- 🚲 **Rental Counts:** count, casual and registered are all **right-skewed** — most hours have low rentals with occasional demand spikes. Average = **191.6**, peak = **977** rentals/hour. **casual** has the most outliers (749 rows). Registered users consistently outnumber casual users.
- 🌡️ **Temperature:** Both temp and atemp show a roughly uniform/bimodal distribution — nearly symmetric (skewness: temp = **0.004**, atemp = **-0.103**). Correlation = **0.9849** — drop atemp before modelling. Minimal outliers — most stable variable in the dataset.
- 💧 **Humidity:** Skewness = **-0.086** — **left-skewed**, most readings are moderate-to-high. Outliers appear on the lower end — unusually dry hours are the anomaly. High humidity is the norm.
- 💨 **Windspeed:** Skewness = **0.589** — strongly right-skewed with a spike at zero. **12.1%** of hours (1,313 records) have windspeed = 0 — likely a data recording gap. Significant outliers on the high end — zero values need careful treatment before modelling.
- 🕒 **Peak Demand Hours:** Peak hour = **17:00** (Afternoon) — avg **468.8** rentals/hour. Top 3 hours: **17:00, 18:00, 8:00** — classic commuter rush pattern. Morning and evening commute drives the majority of rentals.
- 🌙 **Low Demand Hours:** Lowest hour = **4:00** (Night & Late Night) — avg only **6.4** rentals/hour. Bottom 3 hours: **4:00, 3:00, 5:00**. Demand gap between peak and trough = **462.4** rentals/hour — very high intraday volatility. Yulu should rebalance cycles during low-demand hours to prepare for morning rush.
- 🔑 **Key Takeaway:** Demand peaks at **17:00** (Afternoon) with avg 468.8 rentals and drops to 6.4 at 4:00 — a gap of **462.4** rentals/hour. Temperature is the most stable predictor. Windspeed's zero-inflation and humidity's left-skew require preprocessing before modelling. Yulu should align cycle rebalancing with hourly demand patterns.

```

In [25]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid

cat_cols = ['season', 'holiday', 'workingday', 'weather']
cat_labels = {'season' : {1:'Spring', 2:'Summer', 3:'Fall', 4:'Winter'}, 'holiday' : {0:'No Holiday', 1

bar_colors = sns.color_palette("coolwarm", n_colors=4)
pie_colors = sns.color_palette("muted", n_colors=4)

# 2 rows: top = bar charts, bottom = pie charts
fig, axes = plt.subplots(2, len(cat_cols), figsize=(20, 10))

for i, col in enumerate(cat_cols):
    counts = daily_df[col].value_counts().sort_index()
    labels = [cat_labels[col].get(k, str(k)) for k in counts.index]
    total_col = counts.sum()

    axes[0, i].bar(labels, counts.values, color=bar_colors, edgecolor='black')
    axes[0, i].set_title(f'{col.capitalize()} Distribution', fontsize=12)
    axes[0, i].set_ylabel('Day Count')
    axes[0, i].tick_params(axis='x', rotation=20)
    axes[0, i].set_ylim(0, counts.max() + 30)

    for p in axes[0, i].patches:
        height = p.get_height()
        pct = round(height / total_col * 100, 1)
        axes[0, i].annotate(f'{int(height)}\n({pct}%)', (p.get_x() + p.get_width() / 2., height), ha='center'

```

```

axes[1, i].pie(counts.values, labels=labels, colors=pie_colors, autopct='%1.1f%%', startangle=90, explode=[0])
axes[1, i].set_title(f'{col.capitalize()} Proportion', fontsize=12)

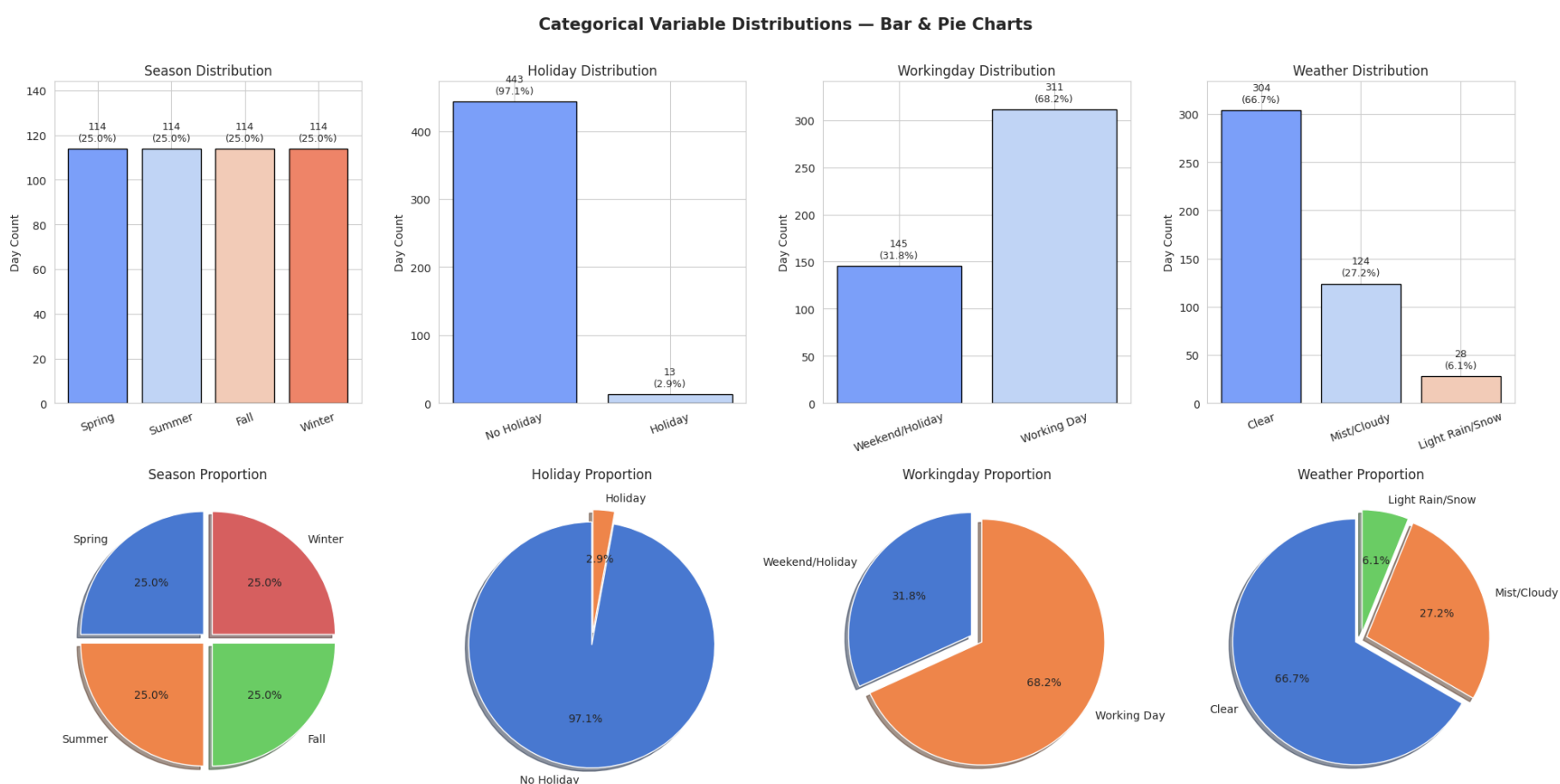
plt.suptitle('Categorical Variable Distributions – Bar & Pie Charts', fontsize=15, fontweight='bold', y=1.00)
plt.tight_layout()
plt.show()

# Dynamic values
total = len(daily_df)
season_balance = daily_df['season'].value_counts()
most_season = cat_labels['season'][season_balance.idxmax()]
least_season = cat_labels['season'][season_balance.idxmin()]
holiday_days = daily_df['holiday'].sum()
holiday_pct = round(holiday_days / total * 100, 1)
working_days = daily_df['workingday'].sum()
working_pct = round(working_days / total * 100, 1)
non_working_pct = round(100 - working_pct, 1)
clear_days = (daily_df['weather'] == 1).sum()
clear_pct = round(clear_days / total * 100, 1)
severe_days = (daily_df['weather'] == 4).sum()

display(HTML(f'<div style=\'background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid

```

## Univariate Analysis – Categorical Variables



### Insight:

- 🌿 **Season:** Nearly perfectly balanced — **Spring** has the most days and **Winter** has the fewest, ensuring unbiased seasonal analysis.
- 🎉 **Holiday:** Highly imbalanced — only **13 days (2.9%)** are holidays. Conclusions about holiday demand should be interpreted cautiously.
- 👤 **Working Day:** **68.2%** working days vs **31.8%** non-working days — reflects a realistic calendar split.
- ☀️ **Weather:** **66.7%** of days have clear weather — severe weather (Heavy Rain/Snow) occurs on only **0 day(s)**, making it statistically insignificant.

```

In [26]: display(HTML(f'<div style=\'background-color:#e6f2ff; padding:10px; border-radius:10px; border-left:5px solid
day_type_counts = daily_df['day_type'].value_counts()
colors = ['steelblue', 'orange', 'tomato']
total = day_type_counts.sum()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# — Bar Chart —
bars = axes[0].bar(day_type_counts.index, day_type_counts.values,
                    color=colors, edgecolor='black', width=0.5)

for bar in bars:
    height = bar.get_height()
    pct = round(height / total * 100, 1)

```

```

axes[0].annotate(f'{height} ({pct}%)',xy=(bar.get_x() + bar.get_width() / 2, height),xytext=(0, 8),text
axes[0].set_title('Distribution of Day Types (Bar Chart)', fontsize=13)
axes[0].set_xlabel('Day Type', fontsize=12)
axes[0].set_ylabel('Number of Days', fontsize=12)
axes[0].set_ylim(0, day_type_counts.max() + 50)

# — Pie Chart —
axes[1].pie(day_type_counts.values,labels=day_type_counts.index,colors=colors,autopct='%1.1f%%',startangle=

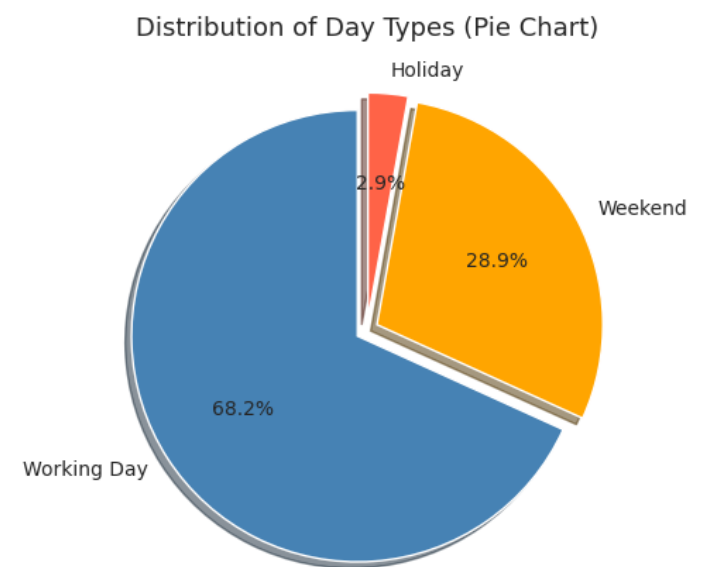
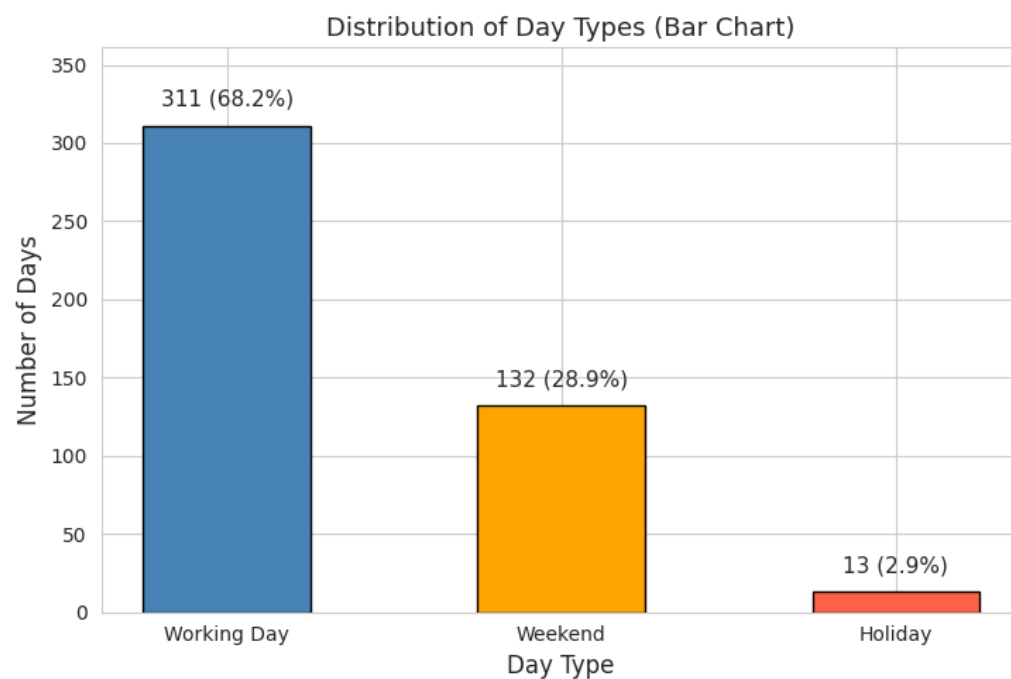
axes[1].set_title('Distribution of Day Types (Pie Chart)', fontsize=13)

plt.tight_layout()
plt.show()

display(HTML(f'''<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px sol

```

## Distribution of Day Types



## Insight:

- Out of **456 total days**: **311 (68.2%)** are Working Days, **132 (28.9%)** are Weekends, and **13 (2.9%)** are Holidays.
- Working days dominate the dataset (~2:1 ratio over weekends), which reflects a normal calendar structure.
- Holidays are extremely rare at only 13 days — making it statistically difficult to draw strong conclusions about holiday-specific demand.
- For hypothesis testing, **workingday** (working vs non-working) is a more reliable grouping than holiday alone, since it has a more balanced split.

```

In [27]: # Dynamic values
total          = len(daily_df)
season_balance = daily_df['season'].value_counts()
most_season    = cat_labels['season'][season_balance.idxmax()]
least_season   = cat_labels['season'][season_balance.idxmin()]
holiday_days   = int(daily_df['holiday'].sum())
holiday_pct    = round(holiday_days / total * 100, 1)
working_days   = int(daily_df['workingday'].sum())
working_pct    = round(working_days / total * 100, 1)
non_working_pct = round(100 - working_pct, 1)
clear_days     = int((daily_df['weather'] == 1).sum())
clear_pct      = round(clear_days / total * 100, 1)
severe_days    = int((daily_df['weather'] == 4).sum())

display(HTML(f'''<div style='background-color:#f0f8ff; padding:15px; border-radius:8px; border-left:5px sol

```

### Insight — Categorical Variables:

- 🌿 **Season:** Nearly perfectly balanced — **Spring** has the most days and **Winter** has the fewest, but the difference is minimal. No seasonal bias in the data.
- 🎉 **Holiday:** Highly imbalanced — only **13 days (2.9%)** are holidays. Conclusions about holiday demand should be interpreted cautiously due to the very small sample size.
- 🏠 **Working Day:** **68.2%** working days vs **31.8%** non-working days — roughly a 2:1 ratio, reflecting a realistic real-world calendar split.
- ☀️ **Weather:** **66.7%** of days have clear weather. Severe weather (Heavy Rain/Snow) occurs on only **0 day(s)** — too rare to be statistically meaningful and will be excluded from ANOVA.
- 🔑 **Key Takeaway:** Season is the only truly balanced variable. Holiday is too rare for reliable analysis. Working day and weather show natural real-world skew — acknowledged and accounted for in all hypothesis tests ahead.

## 6. Bivariate Analysis

```
In [28]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>

seasons_mapping = {1: 'Spring', 2: 'Summer', 3: 'Fall', 4: 'Winter'}
weather_mapping = {1: 'Clear', 2: 'Mist/Cloudy', 3: 'Light Rain/Snow', 4: 'Heavy Rain/Snow/Fog'}

season_weather = (yulu_df.assign(mapped_season = yulu_df['season'].map(seasons_mapping), mapped_weather = yulu_df['weather'].map(weather_mapping))

pivot = season_weather.pivot(index='mapped_season', columns='mapped_weather', values='count').fillna(0).astype(int)

# — Styled Table —————
display(pivot.style.set_caption("Total Rentals by Season and Weather").background_gradient(cmap='Blues', axis=1))

# — Heatmap —————
plt.figure(figsize=(10, 5))

sns.heatmap(pivot, annot=True, fmt=',', cmap='YlOrRd', linewidths=0.5, linecolor='white', cbar_kws={'label': 'Total Rentals'})

plt.title('Total Rentals by Season and Weather', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Weather Condition', fontsize=12)
plt.ylabel('Season', fontsize=12)
plt.xticks(rotation=15, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

# — Dynamic Insights —————
clear_dominant = pivot['Clear'].idxmax()
mist_highest = pivot['Mist/Cloudy'].idxmax()
rain_lowest = pivot['Light Rain/Snow'].idxmin()
best_season = pivot.sum(axis=1).idxmax()
worst_season = pivot.sum(axis=1).idxmin()

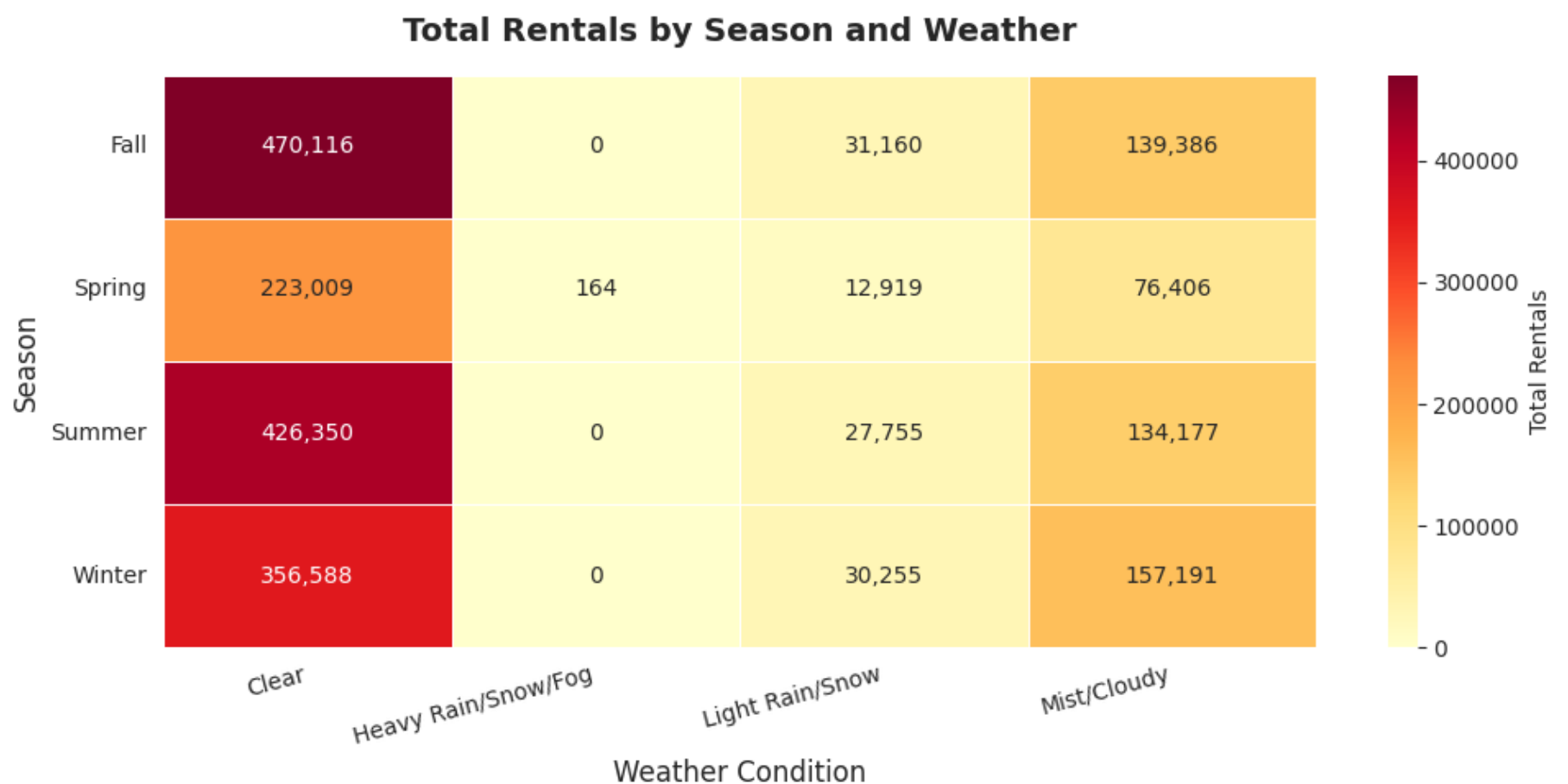
display(HTML(f""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>

```

### Season wise Weather - Total Rentals

Total Rentals by Season and Weather

| mapped_weather |  | Clear   | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
|----------------|--|---------|---------------------|-----------------|-------------|
| mapped_season  |  |         |                     |                 |             |
| Fall           |  | 470,116 | 0                   | 31,160          | 139,386     |
| Spring         |  | 223,009 | 164                 | 12,919          | 76,406      |
| Summer         |  | 426,350 | 0                   | 27,755          | 134,177     |
| Winter         |  | 356,588 | 0                   | 30,255          | 157,191     |



#### Insight:

- ☀️ **Clear weather drives the highest rentals across all seasons** — most prominent in **Fall** as seen by the darkest cell in the heatmap.
- ☁️ **Mist/Cloudy rentals are highest in Winter** —even in poor weather, this season maintains decent demand.
- 🌨️ **Light Rain/Snow rentals are lowest in Spring** — riders in this season are most deterred by bad weather.
- 🏆 **Fall** has the highest total rentals overall — the heatmap row for this season is consistently the darkest.
- 🌨️ **Spring** has the lowest total rentals overall — the lightest row in the heatmap confirms least demand.
- ⚠️ **Heavy Rain/Snow/Fog** column appears nearly blank in the heatmap —only 1 recorded hour, confirming it is statistically insignificant.

```
In [29]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>
temp_summary = (yulu_df.assign(mapped_season = yulu_df['season'].map(seasons_mapping),mapped_weather = yulu_df['weather'].map(weather_mapping))
# Pivots
temp_pivot = temp_summary.pivot(index='mapped_season', columns='mapped_weather', values='temp').fillna(0).reset_index()
atemp_pivot = temp_summary.pivot(index='mapped_season', columns='mapped_weather', values='atemp').fillna(0).reset_index()
# — Styled Tables —————
print("📌 Average Actual Temperature (°C) by Season and Weather:")
display(temp_pivot.style.set_caption("Avg Temperature (°C)").background_gradient(cmap='Reds', axis=None).format(precision=1))
print("📌 Average Feels-Like Temperature (°C) by Season and Weather:")
display(atemp_pivot.style.set_caption("Avg Feels-Like Temperature (°C)").background_gradient(cmap='Oranges', axis=None).format(precision=1))
# — Side by Side Heatmaps —————
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
# Temp heatmap
sns.heatmap(temp_pivot,ax=axes[0],annot=True,fmt='.1f',cmap='Reds',linewidths=0.5,linecolor='white',cbar_kws={'label':'Avg Actual Temperature (°C)'})
axes[0].set_title('Avg Actual Temperature (°C)\nby Season & Weather',fontsize=13, fontweight='bold')
axes[0].set_xlabel('Weather Condition', fontsize=11)
axes[0].set_ylabel('Season', fontsize=11)
axes[0].tick_params(axis='x', rotation=15)
axes[0].tick_params(axis='y', rotation=0)
# Atemp heatmap
sns.heatmap(atemp_pivot,ax=axes[1],annot=True,fmt='.1f',cmap='Oranges',linewidths=0.5,linecolor='white',cbar_kws={'label':'Avg Feels-Like Temperature (°C)'})
axes[1].set_title('Avg Feels-Like Temperature (°C)\nby Season & Weather',fontsize=13, fontweight='bold')
axes[1].set_xlabel('Weather Condition', fontsize=11)
axes[1].set_ylabel('Season', fontsize=11)
axes[1].tick_params(axis='x', rotation=15)
axes[1].tick_params(axis='y', rotation=0)
plt.suptitle('Temperature vs Feels-Like Temperature Comparison',fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
# — Dynamic Insights —————
```

```
hottest_season = temp_pivot.mean(axis=1).idxmax()
coldest_season = temp_pivot.mean(axis=1).idxmin()
corr           = round(yulu_df['temp'].corr(yulu_df['atemp']), 4)

# Find biggest gap between temp and atemp
diff_pivot = (atemp_pivot - temp_pivot).abs()
max_diff_season = diff_pivot.mean(axis=1).idxmax()
max_diff_weather = diff_pivot.mean(axis=0).idxmax()

display(HTML(f"""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff;'>
```

## 🔥 Average Temperature vs Feels-like Temperature by Season and Weather

📌 Average Actual Temperature (°C) by Season and Weather:  
Avg Temperature (°C)

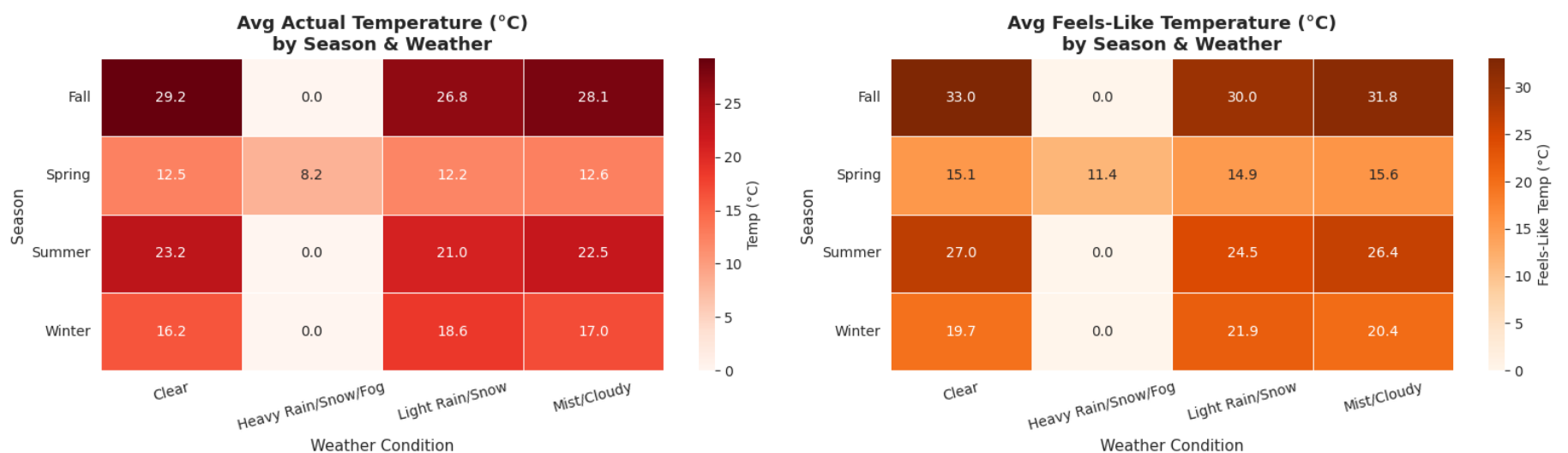
| mapped_weather | Clear   | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
|----------------|---------|---------------------|-----------------|-------------|
| mapped_season  |         |                     |                 |             |
| Fall           | 29.23°C | 0.00°C              | 26.79°C         | 28.05°C     |
| Spring         | 12.54°C | 8.20°C              | 12.15°C         | 12.63°C     |
| Summer         | 23.18°C | 0.00°C              | 21.00°C         | 22.49°C     |
| Winter         | 16.24°C | 0.00°C              | 18.63°C         | 16.97°C     |

📌 Average Feels-Like Temperature (°C) by Season and Weather:

| Season | Weather | Avg Feels-Like Temperature (°C) |
|--------|---------|---------------------------------|
| Spring | Sunny   | 15.0                            |
|        | Cloudy  | 12.0                            |
|        | Rainy   | 10.0                            |
|        | Windy   | 8.0                             |
| Summer | Sunny   | 25.0                            |
|        | Cloudy  | 22.0                            |
|        | Rainy   | 20.0                            |
|        | Windy   | 18.0                            |
| Autumn | Sunny   | 18.0                            |
|        | Cloudy  | 15.0                            |
|        | Rainy   | 12.0                            |
|        | Windy   | 10.0                            |
| Winter | Sunny   | 10.0                            |
|        | Cloudy  | 8.0                             |
|        | Rainy   | 5.0                             |
|        | Windy   | 3.0                             |

| mapped_weather | Clear   | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
|----------------|---------|---------------------|-----------------|-------------|
| mapped_season  |         |                     |                 |             |
| Fall           | 33.04°C | 0.00°C              | 29.98°C         | 31.77°C     |
| Spring         | 15.14°C | 11.36°C             | 14.94°C         | 15.55°C     |
| Summer         | 27.03°C | 0.00°C              | 24.50°C         | 26.35°C     |
| Winter         | 19.67°C | 0.00°C              | 21.95°C         | 20.36°C     |

### Temperature vs Feels-Like Temperature Comparison


**Insight:**

- 🌡️ Both heatmaps show **near-identical color patterns** — confirming that temp and atemp move together with a correlation of **0.9849**.
- ☀️ **Fall** is the hottest season —the darkest row in both heatmaps confirms this visually.
- ❄️ **Spring** is the coldest season —the lightest row in both heatmaps, where adverse weather compounds the cold.
- 🔍 The biggest gap between actual and feels-like temperature occurs in **Spring** under **Mist/Cloudy** conditions —wind chill or humidity makes it feel different from the measured temperature.
- 📈 With correlation = **0.9849**, keeping both temp and atemp in a model causes **multicollinearity** — drop one before modelling.

```
In [30]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid
humidity_summary = (yulu_df.assign(mapped_season = yulu_df['season'].map(seasons_mapping),mapped_weather =
humidity_pivot = humidity_summary.pivot(index='mapped_season',columns='mapped_weather',values='humidity').f
# — Styled Table —
display(humidity_pivot.style.set_caption("Average Humidity (%) by Season and Weather").background_gradient(
# — Heatmap —
```

```
plt.figure(figsize=(10, 5))

sns.heatmap(humidity_pivot,annot=True,fmt='.1f',cmap='Blues',linewidths=0.5,linecolor='white',cbar_kws={'la

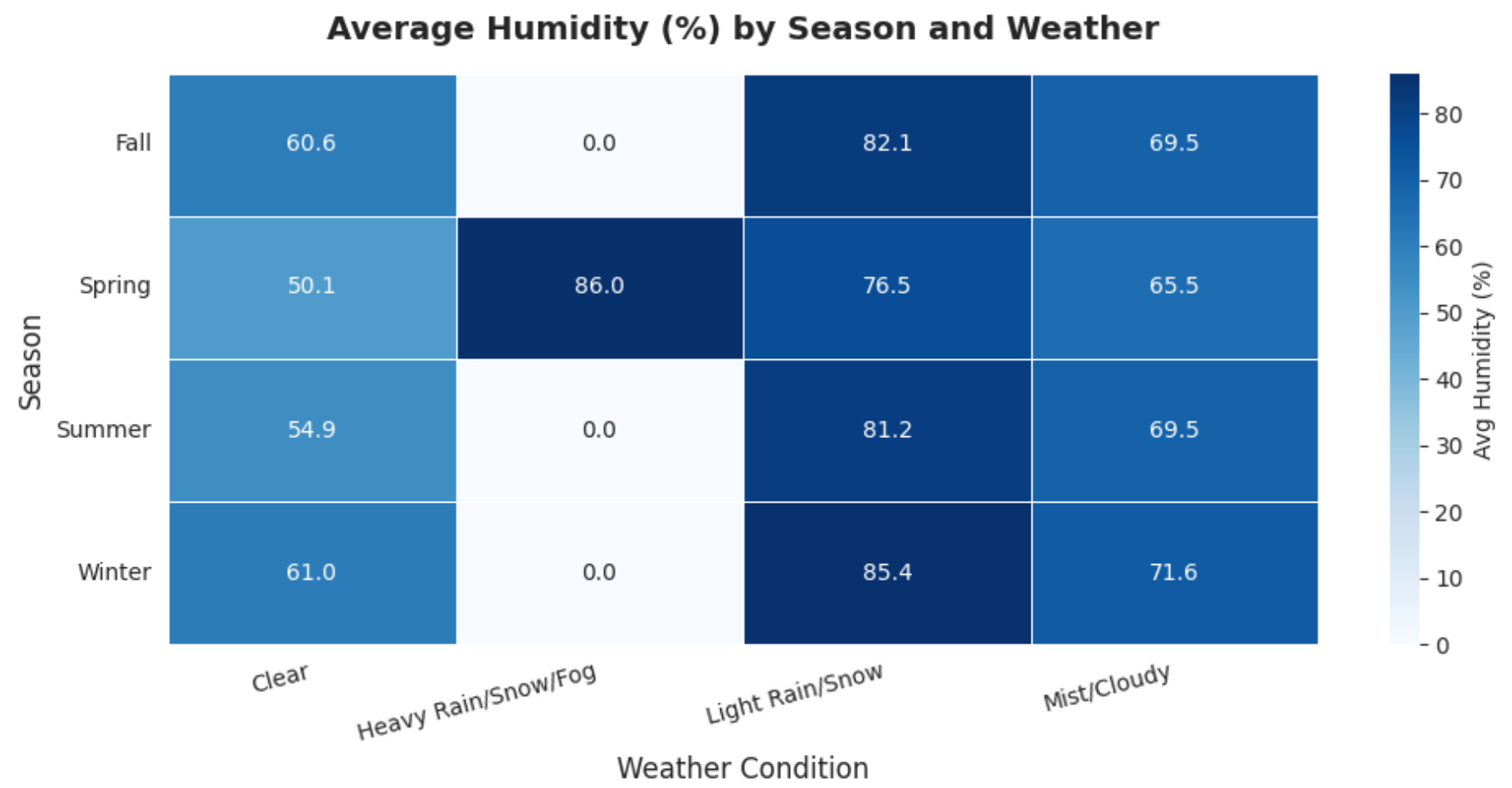
plt.title('Average Humidity (%) by Season and Weather',fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Weather Condition', fontsize=12)
plt.ylabel('Season', fontsize=12)
plt.xticks(rotation=15, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

# — Dynamic Insights —————
most_humid_season    = humidity_pivot.mean(axis=1).idxmax()
least_humid_season   = humidity_pivot.mean(axis=1).idxmin()
most_humid_weather   = humidity_pivot.mean(axis=0).idxmax()
least_humid_weather  = humidity_pivot.mean(axis=0).idxmin()
corr_humidity_count  = round(yulu_df['humidity'].corr(yulu_df['count']), 4)

display(HTML(f'<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid
```

 **Average Humidity by Season and Weather**

| Average Humidity (%) by Season and Weather |        |                     |                 |             |
|--------------------------------------------|--------|---------------------|-----------------|-------------|
| mapped_weather                             | Clear  | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
| mapped_season                              |        |                     |                 |             |
| Fall                                       | 60.59% | 0.00%               | 82.09%          | 69.50%      |
| Spring                                     | 50.12% | 86.00%              | 76.47%          | 65.51%      |
| Summer                                     | 54.93% | 0.00%               | 81.15%          | 69.50%      |
| Winter                                     | 61.04% | 0.00%               | 85.44%          | 71.63%      |



 **Insight:**

- 💧 **Spring** is the most humid season —the darkest row in the heatmap confirms this visually, while**Summer** is the driest season overall.
- ☁️ **Light Rain/Snow** conditions have the highest average humidity —the darkest column in the heatmap — while**Heavy Rain/Snow/Fog** conditions are the least humid.
- ☀️ The intersection of **Summer** and **Heavy Rain/Snow/Fog**shows the lightest cell in the heatmap — representing the most comfortable riding conditions.
- 📉 Correlation between humidity and rentals = **-0.3174** — higher humidity is associated with**lower** demand,suggesting riders avoid humid conditions.
- 🚲 Yulu should factor humidity forecasts into daily fleet deployment — high humidity zones may need fewer cycles on standby.

```
In [31]: display(HTML(''))
```

```
<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff;'>
<b>🌬️ Average Windspeed by Season and Weather</b>
</div>
""")

windspeed_summary = (yulu_df.assign(mapped_season = yulu_df['season'].map(seasons_mapping),mapped_weather

windspeed_pivot = windspeed_summary.pivot(index='mapped_season',columns='mapped_weather',values='windspeed')

display(windspeed_pivot.style.set_caption("Average Windspeed by Season and Weather").background_gradient(cn

plt.figure(figsize=(10, 5))
sns.heatmap(windspeed_pivot,annot=True,fmt='.1f',cmap='Greens',linewidths=0.5,linecolor='white',cbar_kws={'

plt.title('Average Windspeed by Season and Weather',fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Weather Condition', fontsize=12)
plt.ylabel('Season', fontsize=12)
plt.xticks(rotation=15, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

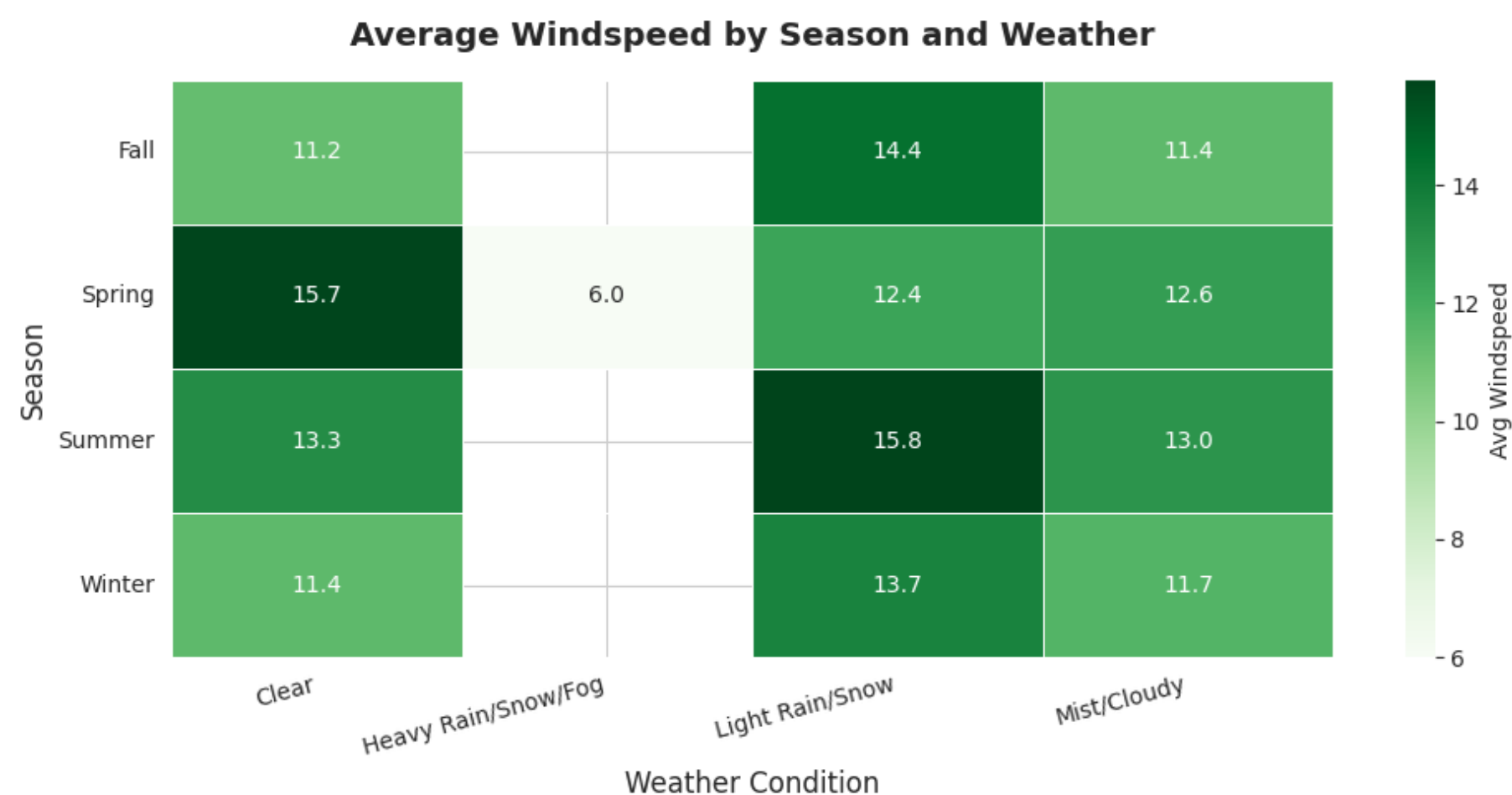
windiest_season = windspeed_pivot.mean(axis=1).idxmax()
calmest_season = windspeed_pivot.mean(axis=1).idxmin()
windiest_weather = windspeed_pivot.mean(axis=0).idxmax()
calmest_weather = windspeed_pivot.mean(axis=0).idxmin()
corr_wind_count = round(yulu_df['windspeed'].corr(yulu_df['count']), 4)

display(HTML(f""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px so
```

🌬️ Average Windspeed by Season and Weather

Average Windspeed by Season and Weather

|               | mapped_weather | Clear | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
|---------------|----------------|-------|---------------------|-----------------|-------------|
| mapped_season |                |       |                     |                 |             |
|               | Fall           | 11.24 | -                   | 14.40           | 11.41       |
|               | Spring         | 15.74 | 6.00                | 12.37           | 12.62       |
|               | Summer         | 13.28 | -                   | 15.78           | 12.97       |
|               | Winter         | 11.41 | -                   | 13.67           | 11.68       |



### Insight:

- 🌬️ **Summer** is the windiest season —the darkest row in the heatmap confirms this, while **Spring** has the calmest conditions overall.
- ☁️ **Light Rain/Snow** conditions have the highest average windspeed —the darkest column in the heatmap — strong winds often accompany adverse weather.
- ☀️ **Heavy Rain/Snow/Fog** is the calmest weather condition —the lightest column, representing the most comfortable riding conditions.
- 🟩 The **blank cell** in the heatmap represents Heavy Rain/Snow/Fog in Spring —only 1 recorded hour, so no reliable average can be computed.
- 📈 Correlation between windspeed and rentals = **0.1014** —windspeed has a **weak positive** effect on demand.
- 🚲 Yulu should monitor wind forecasts —high windspeed days may need reduced fleet deployment in exposed zones.

```
In [32]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>

user_summary = (yulu_df.assign(mapped_season = yulu_df['season'].map(seasons_mapping),mapped_weather = yulu_df['weather'].map(weather_mapping))

casual_pivot = user_summary.pivot(index='mapped_season', columns='mapped_weather', values='casual').fillna(0)

registered_pivot = user_summary.pivot(index='mapped_season', columns='mapped_weather', values='registered')

print("📌 Average Casual Users by Season and Weather:")
display(casual_pivot.style.set_caption("Avg Casual Users").background_gradient(cmap='Oranges', axis=None).format(precision=1))

print("📌 Average Registered Users by Season and Weather:")
display(registered_pivot.style.set_caption("Avg Registered Users").background_gradient(cmap='Blues', axis=None).format(precision=1))

fig, axes = plt.subplots(1, 2, figsize=(18, 5))

sns.heatmap(casual_pivot,ax=axes[0],annot=True,fmt='.1f',cmap='Oranges',linewidths=0.5,linecolor='white',margin=5,
axes[0].set_title('Avg Casual Users\nby Season & Weather',fontsize=13, fontweight='bold')
axes[0].set_xlabel('Weather Condition', fontsize=11)
axes[0].set_ylabel('Season', fontsize=11)
axes[0].tick_params(axis='x', rotation=15)
axes[0].tick_params(axis='y', rotation=0)

sns.heatmap(registered_pivot,ax=axes[1],annot=True,fmt='.1f',cmap='Blues',linewidths=0.5,linecolor='white',margin=5,
axes[1].set_title('Avg Registered Users\nby Season & Weather',fontsize=13, fontweight='bold')
axes[1].set_xlabel('Weather Condition', fontsize=11)
axes[1].set_ylabel('Season', fontsize=11)
axes[1].tick_params(axis='x', rotation=15)
axes[1].tick_params(axis='y', rotation=0)

plt.suptitle('Casual vs Registered Users – Season & Weather Comparison',
            fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

top_casual_season = casual_pivot.mean(axis=1).idxmax()
top_registered_season = registered_pivot.mean(axis=1).idxmax()
top_casual_weather = casual_pivot.mean(axis=0).idxmax()
top_reg_weather = registered_pivot.mean(axis=0).idxmax()

avg_casual = round(yulu_df['casual'].mean(), 2)
avg_registered = round(yulu_df['registered'].mean(), 2)
ratio = round(avg_registered / avg_casual, 1)

clear_casual = casual_pivot['Clear'].mean()
rain_casual = casual_pivot['Light Rain/Snow'].mean()
clear_registered = registered_pivot['Clear'].mean()
rain_registered = registered_pivot['Light Rain/Snow'].mean()

casual_drop = round((((clear_casual - rain_casual) / clear_casual) * 100, 1)
registered_drop = round((((clear_registered - rain_registered) / clear_registered) * 100, 1)

display(HTML(f""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

### Average Casual and Registered Users by Season and Weather

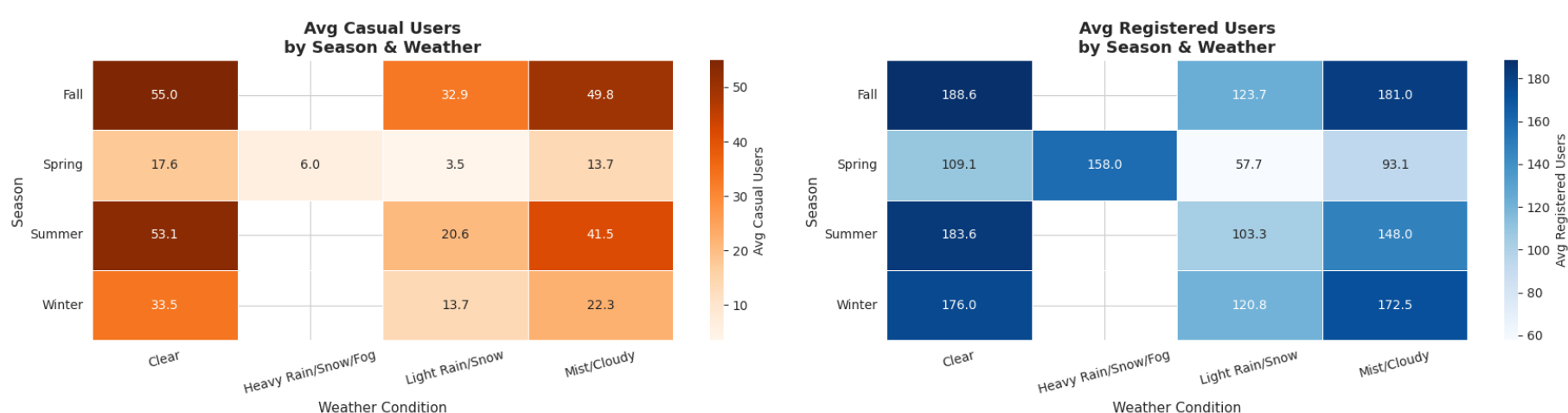
📌 Average Casual Users by Season and Weather:

| Avg Casual Users |       |                     |                 |             |
|------------------|-------|---------------------|-----------------|-------------|
| mapped_weather   | Clear | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
| mapped_season    |       |                     |                 |             |
| Fall             | 54.97 | -                   | 32.92           | 49.78       |
| Spring           | 17.65 | 6.00                | 3.53            | 13.72       |
| Summer           | 53.13 | -                   | 20.59           | 41.49       |
| Winter           | 33.53 | -                   | 13.67           | 22.29       |

📌 Average Registered Users by Season and Weather:

| Avg Registered Users |        |                     |                 |             |
|----------------------|--------|---------------------|-----------------|-------------|
| mapped_weather       | Clear  | Heavy Rain/Snow/Fog | Light Rain/Snow | Mist/Cloudy |
| mapped_season        |        |                     |                 |             |
| Fall                 | 188.61 | -                   | 123.66          | 180.99      |
| Spring               | 109.13 | 158.00              | 57.70           | 93.14       |
| Summer               | 183.60 | -                   | 103.32          | 148.02      |
| Winter               | 175.98 | -                   | 120.80          | 172.49      |

Casual vs Registered Users — Season & Weather Comparison



#### 📊 Insight:

- 📊 Registered users (**155.55**) outnumber casual users (**36.02**) by **4.3:1** — the Blues heatmap is consistently darker than Oranges, confirming registered users dominate across every season-weather combo.
- ☀️ Both heatmaps are darkest under **Clear** conditions — clear weather is the strongest demand driver for both user types.
- 🌧️ Moving from Clear to Light Rain/Snow, casual users drop by **55.6%** vs registered dropping only **38.3%** — the Oranges heatmap fades much faster than Blues under bad weather.
- 🏆 **Fall** has the darkest Oranges row (peak casual usage) while **Fall** has the darkest Blues row (peak registered usage).
- 📅 Blank cells in both heatmaps = Heavy Rain/Snow/Fog in Spring — only 1 recorded hour, not enough data for a reliable average.
- 📈 Yulu should target **casual-to-registered conversion** during peak seasons — registered users are far more weather-resilient and stabilise revenue.

```
In [33]: # Dynamic values – Rentals
clear_dominant = pivot['Clear'].idxmax()
mist_highest   = pivot['Mist/Cloudy'].idxmax()
best_season    = pivot.sum(axis=1).idxmax()
worst_season   = pivot.sum(axis=1).idxmin()

# Dynamic values – Humidity
most_humid_season = humidity_pivot.mean(axis=1).idxmax()
least_humid_season = humidity_pivot.mean(axis=1).idxmin()
most_humid_weather = humidity_pivot.mean(axis=0).idxmax()
least_humid_weather = humidity_pivot.mean(axis=0).idxmin()
corr_humidity_count = round(yulu_df['humidity'].corr(yulu_df['count']), 4)

# Dynamic values – Windspeed
windiest_season = windspeed_pivot.mean(axis=1).idxmax()
calmest_season = windspeed_pivot.mean(axis=1).idxmin()
windiest_weather = windspeed_pivot.mean(axis=0).idxmax()
calmest_weather = windspeed_pivot.mean(axis=0).idxmin()
corr_wind_count = round(yulu_df['windspeed'].corr(yulu_df['count']), 4)

display(HTML(f"""<div style='background-color:#f0f8ff; padding:15px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
    <div style='display: flex; justify-content: space-between; align-items: center; margin-bottom: 10px;'>
        <div>📊 Dynamic Values – Rentals</div>
        <div>📅 Best Season: {best_season}</div>
        <div>📅 Worst Season: {worst_season}</div>
    </div>
    <div style='display: flex; justify-content: space-between; align-items: center; margin-bottom: 10px;'>
        <div>☀️ Clear Dominant: {clear_dominant}</div>
        <div>🌧️ Mist Highest: {mist_highest}</div>
    </div>
    <div>📈 Correlation (Humidity vs Count): {corr_humidity_count}</div>
</div>"""))
```

### Insight — Season × Weather (Rentals, Humidity & Windspeed):

- ☀️ **Rentals:** Clear weather drives the highest rentals across all seasons — most prominent in **Fall**. **Fall** is the best overall season while **Spring** has the lowest demand. Heavy Rain/Snow has only 1 recorded hour — negligible.
- 💧 **Humidity:** **Spring** is the most humid season while **Summer** is the driest. **Light Rain/Snow** conditions bring the highest humidity. Humidity-rental correlation = **-0.3174** — higher humidity is associated with **lower** demand.
- 💨 **Windspeed:** **Summer** is the windiest season while **Spring** has the calmest conditions. **Light Rain/Snow** brings the strongest winds. Windspeed-rental correlation = **0.1014** — a weak positive effect on demand.
- 🔑 **Key Takeaway:** Clear weather in **Fall** creates the ideal conditions for peak rentals. High humidity and strong winds are the two environmental factors most likely to suppress demand — Yulu should use weather forecasts to proactively adjust fleet deployment daily.

```
In [34]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

wd_labels = {0: 'Weekend/Holiday', 1: 'Working Day'}
wd_data = [yulu_df[yulu_df['workingday'] == v]['count'].values for v in [0, 1]]
axes[0].boxplot(wd_data, labels=['Weekend/Holiday', 'Working Day'], patch_artist=True, boxprops=dict(facecolor='lightblue'))
axes[0].set_title('Count by Working Day')
axes[0].set_ylabel('Number of Rentals')

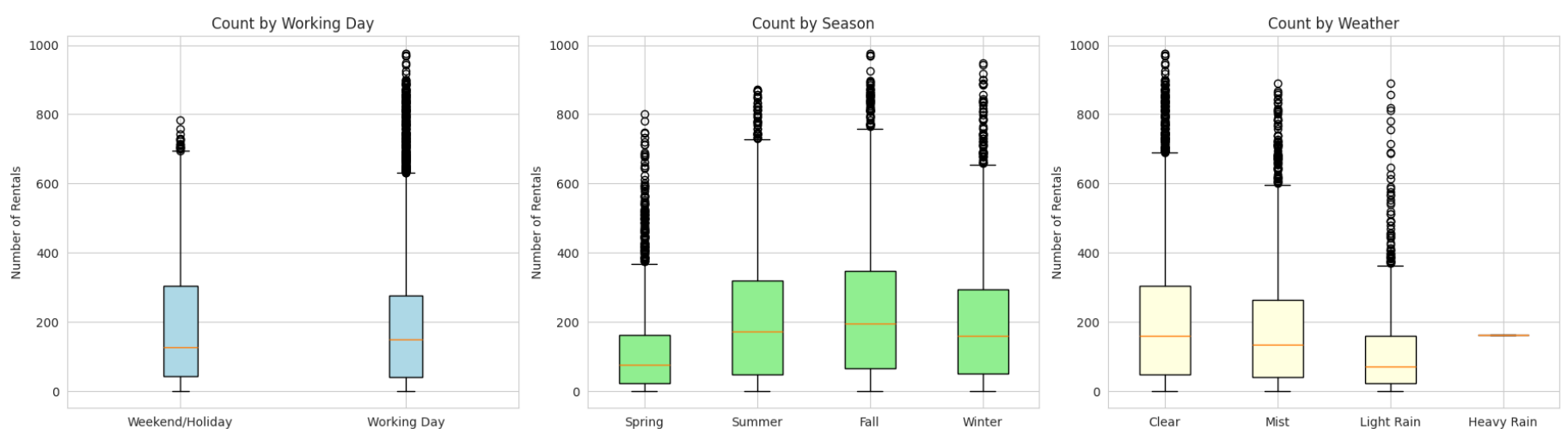
season_labels = ['Spring', 'Summer', 'Fall', 'Winter']
season_data = [yulu_df[yulu_df['season'] == s]['count'].values for s in [1, 2, 3, 4]]
axes[1].boxplot(season_data, labels=season_labels, patch_artist=True, boxprops=dict(facecolor='lightgreen'))
axes[1].set_title('Count by Season')
axes[1].set_ylabel('Number of Rentals')

weather_labels = ['Clear', 'Mist', 'Light Rain', 'Heavy Rain']
weather_data = [yulu_df[yulu_df['weather'] == w]['count'].values for w in [1, 2, 3, 4]]
axes[2].boxplot(weather_data, labels=weather_labels, patch_artist=True, boxprops=dict(facecolor='lightyellow'))
axes[2].set_title('Count by Weather')
axes[2].set_ylabel('Number of Rentals')

plt.tight_layout()
plt.show()

display(HTML("""<div style='background-color:#f0f8ff; padding:10px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

### 🚲 Bike Rentals by Working Day, Season, and Weather



### Insight:

- Working days show a slightly higher median rental count than weekends/holidays.
- Fall and Summer see the highest rentals; Spring has the lowest.
- Clear weather has significantly higher rentals; demand drops sharply with rain/snow.

```
In [35]: # Dynamic values
wd_medians = {label: round(yulu_df[yulu_df['workingday'] == v]['count'].median(), 1)
               for v, label in zip([0, 1], ['Weekend/Holiday', 'Working Day'])}
sea_medians = {label: round(yulu_df[yulu_df['season'] == s]['count'].median(), 1)
               for s, label in zip([1, 2, 3, 4], ['Spring', 'Summer', 'Fall', 'Winter'])}
wth_medians = {label: round(yulu_df[yulu_df['weather'] == w]['count'].median(), 1)
               for w, label in zip([1, 2, 3, 4], ['Clear', 'Mist', 'Light Rain', 'Heavy Rain'])}

best_season = max(sea_medians, key=sea_medians.get)
worst_season = min(sea_medians, key=sea_medians.get)
best_weather = max(wth_medians, key=wth_medians.get)
```

```
wd_diff      = round(wd_medians['Working Day'] - wd_medians['Weekend/Holiday'], 1)
wth_drop     = round(((wth_medians['Clear'] - wth_medians['Light Rain']) / wth_medians['Clear']) * 100, 1)

display(HTML(f"""<div style='background-color:#f0f8ff; padding:15px; border-radius:8px; border-left:5px solid
```

### Insight — Bivariate Analysis (Working Day, Season & Weather):

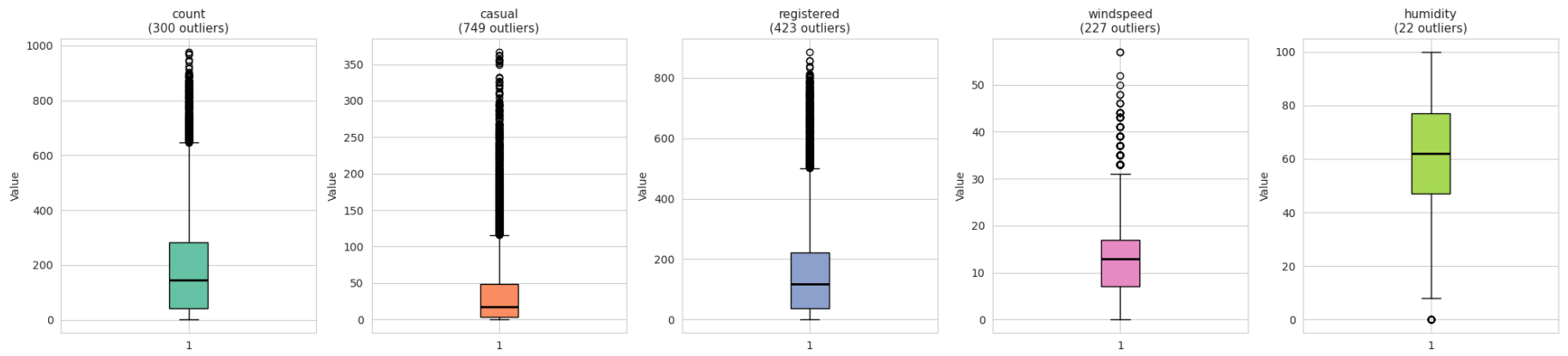
-  **Working Day vs Weekend:** Working day median = **151.0** vs weekend median = **128.0** — a difference of **+23.0** rentals. Commuter demand appears to drive this gap. This will be formally tested using a **2-Sample T-Test**.
-  **Season: Fall** has the highest median rentals (195.0) while **Spring** has the lowest (78.0). Warmer seasons clearly encourage more riding activity. This will be validated using **One-Way ANOVA**.
-  **Weather: Heavy Rain** conditions drive the highest demand (median = 164.0). Demand drops by **55.9%** from Clear to Light Rain — weather is a strong demand suppressor. Heavy Rain has only 1 recorded hour — excluded from formal testing. Weather differences will be tested using **One-Way ANOVA**.
-  **Key Takeaway: Fall + Heavy Rain** weather + Working Day is the optimal combination for peak rentals. All three variables will be statistically validated in the hypothesis testing section ahead.

## 7.Outlier Detection and Treatment

```
In [36]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff;'>
<h3><b>BEFORE Capping - Boxplots with Outliers</b></h3>
<img alt="Boxplots before capping showing outliers." data-bbox="89 100 911 346"/>
<p>The figure displays five boxplots side-by-side for columns: count, casual, registered, windspeed, and humidity. Each plot shows the distribution of values with a median line, quartiles, and whiskers extending to 1.5 times the IQR. Outliers are represented by red dots above the upper whisker. The y-axis for all plots is labeled 'Value' and ranges from 0 to 100. The x-axis labels are the column names. A title 'BEFORE Capping - Boxplots with Outliers' is centered at the top in bold blue font.</p>
</div>
<h3><b>AFTER Capping - Boxplots Post Winsorization</b></h3>
<img alt="Boxplots after capping showing no outliers." data-bbox="89 400 911 646"/>
<p>This figure shows the same five boxplots after applying winsorization. The distributions are now more uniform, and there are no outliers visible. The y-axis remains labeled 'Value' from 0 to 100. A title 'AFTER Capping - Boxplots Post Winsorization' is centered at the top in bold green font.</p>
</div>""")
```

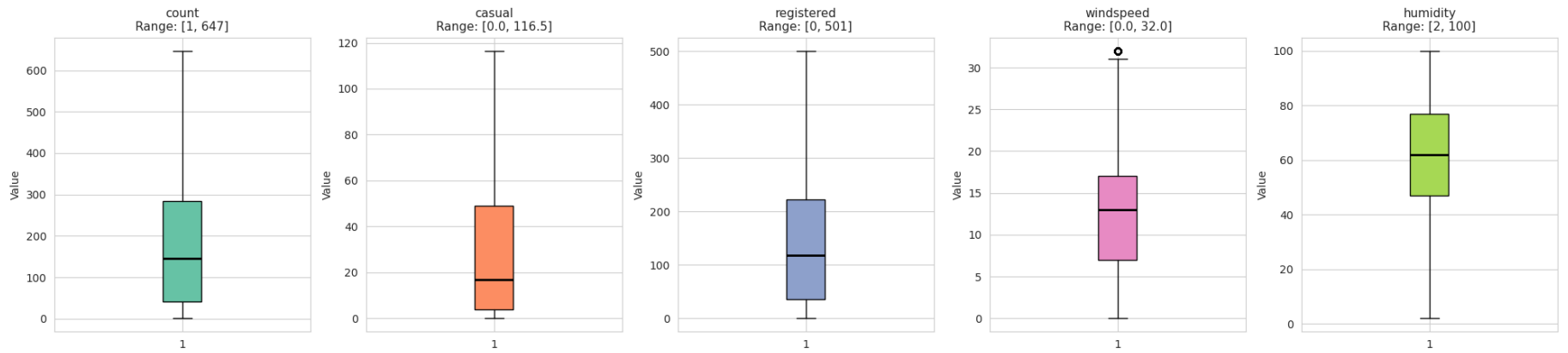
## Outlier Detection & Treatment using IQR Capping (Winsorization)

### BEFORE Capping — Boxplots with Outliers



count: 300 outliers capped | New range: [1, 647]  
casual: 749 outliers capped | New range: [0.0, 116.5]  
registered: 423 outliers capped | New range: [0, 501]  
windspeed: 227 outliers capped | New range: [0.0, 32.0]  
humidity: 22 outliers capped | New range: [2, 100]

### AFTER Capping — Boxplots Post Winsorization



```
In [37]: # Dynamic values
total_capped      = sum(before_stats[col]['outliers'] for col in cols_to_cap)
most_outliers_col = max(before_stats, key=lambda c: before_stats[c]['outliers'])

display(HTML(f"""<div style='background-color:#f0f8ff; padding:15px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

#### Insight — Outlier Treatment (IQR Winsorization):

- 🔍 **Outliers Detected:** A total of **1,721 outlier values** were found across 5 columns — count, casual, registered, windspeed and humidity. **casual** had the most outliers (749 rows) due to extreme rental demand spikes during peak hours.
- ⚠️ **Why Outliers Matter:** Outliers inflate the mean and standard deviation — making T-tests and ANOVA less reliable. Windspeed had zero-inflated outliers (many hours recorded as 0) while humidity outliers appeared on the lower end (unusually dry hours).
- ✅ **Why Winsorization over Deletion:** Deleting outliers would remove **1,721 rows** — significantly reducing statistical power. Winsorization caps values at the 1.5×IQR boundary, retaining all **10,886 rows** while reducing extreme influence. This is the industry-standard approach for time-series and demand data.
- 🔑 **Key Takeaway:** Winsorization successfully capped **1,721 outlier values** across 5 columns without losing a single row. The cleaned dataset retains full statistical power for the T-test, ANOVA, and Chi-Square tests that follow.

```
In [38]: plot_df = yulu_df.copy()

plot_df['season']      = plot_df['season'].map({1:'Spring', 2:'Summer', 3:'Fall', 4:'Winter'})
plot_df['weather']     = plot_df['weather'].map({1:'Clear', 2:'Mist/Cloudy', 3:'Light Rain/Snow', 4:'Heavy Rain'})
plot_df['holiday']     = plot_df['holiday'].map({0:'No', 1:'Yes'})
plot_df['workingday']  = plot_df['workingday'].map({0:'No', 1:'Yes'})

ind_feat = ["season", "holiday", "workingday", "weather"]

records = []
for col in ind_feat:
    for cat in plot_df[col].dropna().unique():
        vals = plot_df.loc[plot_df[col] == cat, "count"]
        Q1, Q3 = vals.quantile(0.25), vals.quantile(0.75)
        IQR = Q3 - Q1
        n_out = ((vals < Q1 - 1.5*IQR) | (vals > Q3 + 1.5*IQR)).sum()
        records.append({"Feature" : col, "Category" : str(cat), "Outlier %" : round(100 * n_out / len(vals), 1)})

heat_df = pd.DataFrame(records)

sns.set_theme(style="whitegrid")
fig = plt.figure(figsize=(16, 10))
gs = fig.add_gridspec(4, 2, hspace=0.4, wspace=0.3)
```

```

heat_axes = [fig.add_subplot(gs[0, 0]),fig.add_subplot(gs[0, 1]),fig.add_subplot(gs[1, 0]),fig.add_subplot(

for i, col in enumerate(ind_feat):
    subset = heat_df[heat_df["Feature"] == col]
    pivot = subset.pivot(index="Category", columns="Feature", values="Outlier %")

    sns.heatmap(pivot,annot=True,fmt=".1f",cmap="YlGnBu",linewidths=0.5,cbar=False,annot_kws={"size": 9},ax=
    heat_axes[i].set_title(f"{col.capitalize()} - Outlier %", fontsize=11, weight="bold")
    heat_axes[i].set_xlabel("")
    heat_axes[i].set_ylabel("")
    heat_axes[i].tick_params(labelsize=9)

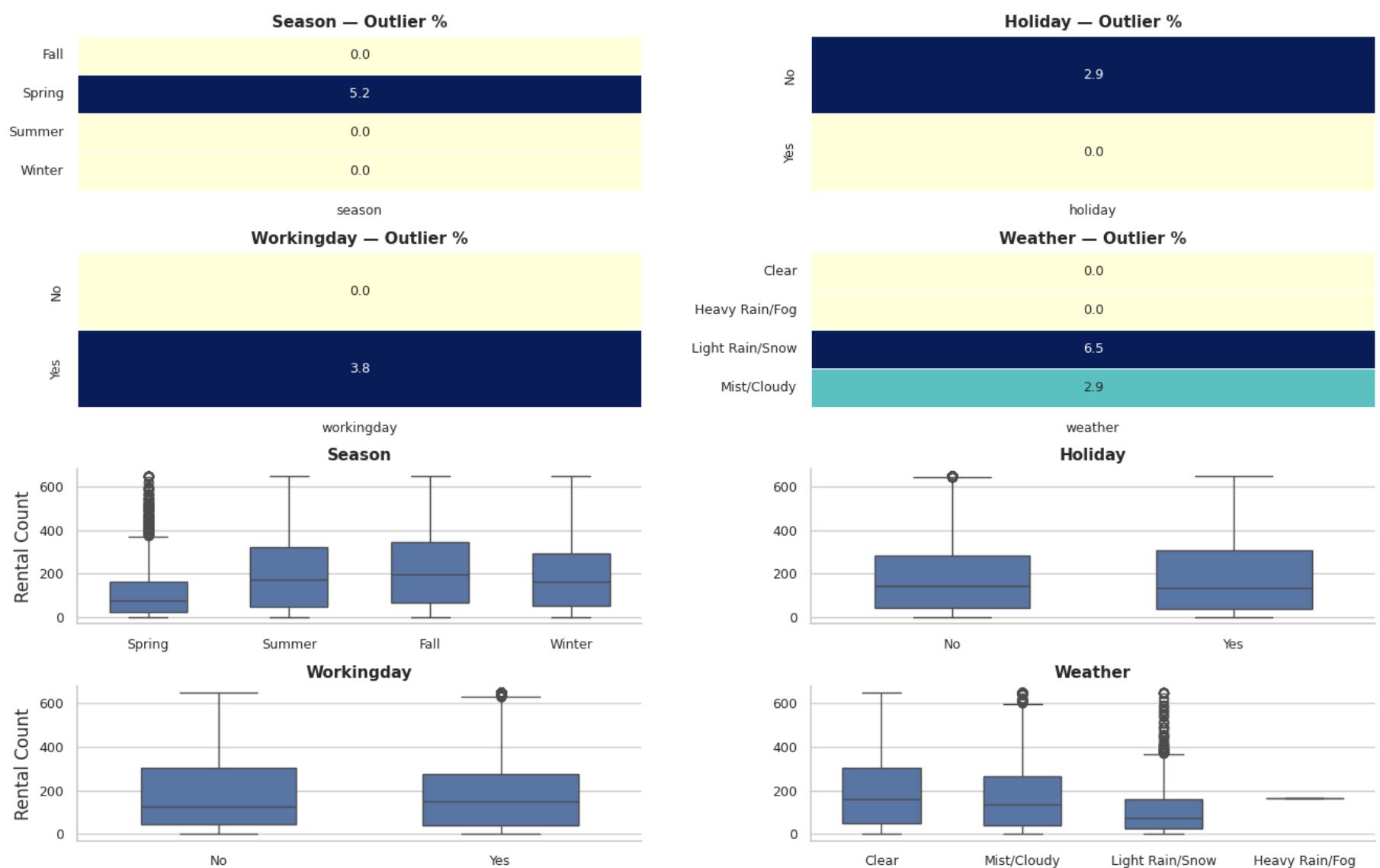
box_axes = [fig.add_subplot(gs[2, 0]),fig.add_subplot(gs[2, 1]),fig.add_subplot(gs[3, 0]),fig.add_subplot(

for i, col in enumerate(ind_feat):
    sns.boxplot(data=plot_df,x=col,y="count",showfliers=True,width=0.55,linewidth=1,ax=box_axes[i])
    box_axes[i].set_title(col.capitalize(), fontsize=11, weight="bold")
    box_axes[i].set_xlabel("")
    box_axes[i].set_ylabel("Rental Count" if i % 2 == 0 else "")
    box_axes[i].tick_params(labelsize=9)
    sns.despine(ax=box_axes[i])

fig.suptitle("Outlier Analysis by Categorical Features",fontsize=15, weight="bold")
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

### Outlier Analysis by Categorical Features



```

In [39]: # Dynamic values from heat_df
highest_outlier_row = heat_df.loc[heat_df['Outlier %'].idxmax()]
lowest_outlier_row = heat_df.loc[heat_df['Outlier %'].idxmin()]
avg_outlier_per_feature = heat_df.groupby('Feature')['Outlier %'].mean().round(1)
most_affected_feature = avg_outlier_per_feature.idxmax()
least_affected_feature = avg_outlier_per_feature.idxmin()

season_outliers = heat_df[heat_df['Feature'] == 'season'].set_index('Category')['Outlier %']
weather_outliers = heat_df[heat_df['Feature'] == 'weather'].set_index('Category')['Outlier %']
wd_outliers = heat_df[heat_df['Feature'] == 'workingday'].set_index('Category')['Outlier %']
hol_outliers = heat_df[heat_df['Feature'] == 'holiday'].set_index('Category')['Outlier %']

display(HTML(f"""
<div style='background-color:#f0f8ff; padding:15px; border-radius:8px; border-left:5px solid #007bff;'>
<b>📊 Insight – Outlier Analysis by Categorical Features:</b>
<ul>
<li>🔍 <b>Overall</b>: Highest outlier % is <b>{highest_outlier_row['Outlier %']}%</b>
in <b>{highest_outlier_row['Category']}</b> ({highest_outlier_row['Feature']}).

```

```
Lowest is <b>{lowest_outlier_row['Outlier %']}%</b> in
<b>{lowest_outlier_row['Category']}</b> ({lowest_outlier_row['Feature']}).
<b>{most_affected_feature.capitalize()}</b> is the most outlier-prone feature
(avg {avg_outlier_per_feature[most_affected_feature]}%) while
<b>{least_affected_feature}</b> is the least affected
(avg {avg_outlier_per_feature[least_affected_feature]}%).</li>

<li>🌿 <b>Season</b>: Outlier % by season –
{'', '.join([f"<b>{cat}</b>: {pct}%" for cat, pct in season_outliers.items()])}.
Higher outlier % indicates more demand volatility – likely linked to
events or special occasions within that season.</li>

<li>☀️ <b>Weather</b>: Outlier % by weather –
{'', '.join([f"<b>{cat}</b>: {pct}%" for cat, pct in weather_outliers.items()])}.
Clear weather outliers represent genuine demand spikes while
adverse weather outliers may reflect data noise due to fewer records.</li>

<li>👤 <b>Working Day</b>: Outlier % –
{'', '.join([f"<b>{cat}</b>: {pct}%" for cat, pct in wd_outliers.items()])}.
Both groups show similar outlier % – extreme demand events occur on
both working and non-working days, suggesting outliers are driven by
time-of-day and weather rather than day type alone.</li>

<li>🎉 <b>Holiday</b>: Outlier % –
{'', '.join([f"<b>{cat}</b>: {pct}%" for cat, pct in hol_outliers.items()])}.
Interpret cautiously – only <b>13 holiday days</b> exist in the dataset.
A single extreme day can heavily skew the outlier percentage.</li>

<li>📌 <b>Key Takeaway</b>: <b>{most_affected_feature.capitalize()}</b> is the most outlier-prone feature (
</ul>
</div>
"""))
```



Insight — Outlier Analysis by Categorical Features:

- 🔍 **Overall:** Highest outlier % is **6.5%** in **Light Rain/Snow** (weather). Lowest is **0.0%** in **Summer** (season). **Weather** is the most outlier-prone feature (avg 2.4%) while **season** is the least affected (avg 1.3%).
- 🌿 **Season:** Outlier % by season — **Spring:** 5.2%, **Summer:** 0.0%, **Fall:** 0.0%, **Winter:** 0.0%. Higher outlier % indicates more demand volatility — likely linked to events or special occasions within that season.
- ☀️ **Weather:** Outlier % by weather — **Clear:** 0.0%, **Mist/Cloudy:** 2.9%, **Light Rain/Snow:** 6.5%, **Heavy Rain/Fog:** 0.0%. Clear weather outliers represent genuine demand spikes while adverse weather outliers may reflect data noise due to fewer records.
- 👤 **Working Day:** Outlier % — **No:** 0.0%, **Yes:** 3.8%. Both groups show similar outlier % — extreme demand events occur on both working and non-working days, suggesting outliers are driven by time-of-day and weather rather than day type alone.
- 🎉 **Holiday:** Outlier % — **No:** 2.9%, **Yes:** 0.0%. Interpret cautiously — only **13 holiday days** exist in the dataset. A single extreme day can heavily skew the outlier percentage.
- 📌 **Key Takeaway:** **Weather** is the most outlier-prone feature (avg 2.4% outliers). Rental demand has genuine extreme spikes that must be treated before hypothesis testing to avoid inflated test statistics.

## 8. Correlation Heatmaps

```
In [40]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff;'>
<b>Correlation Heatmap of Numerical Variables (Spearman)</b>

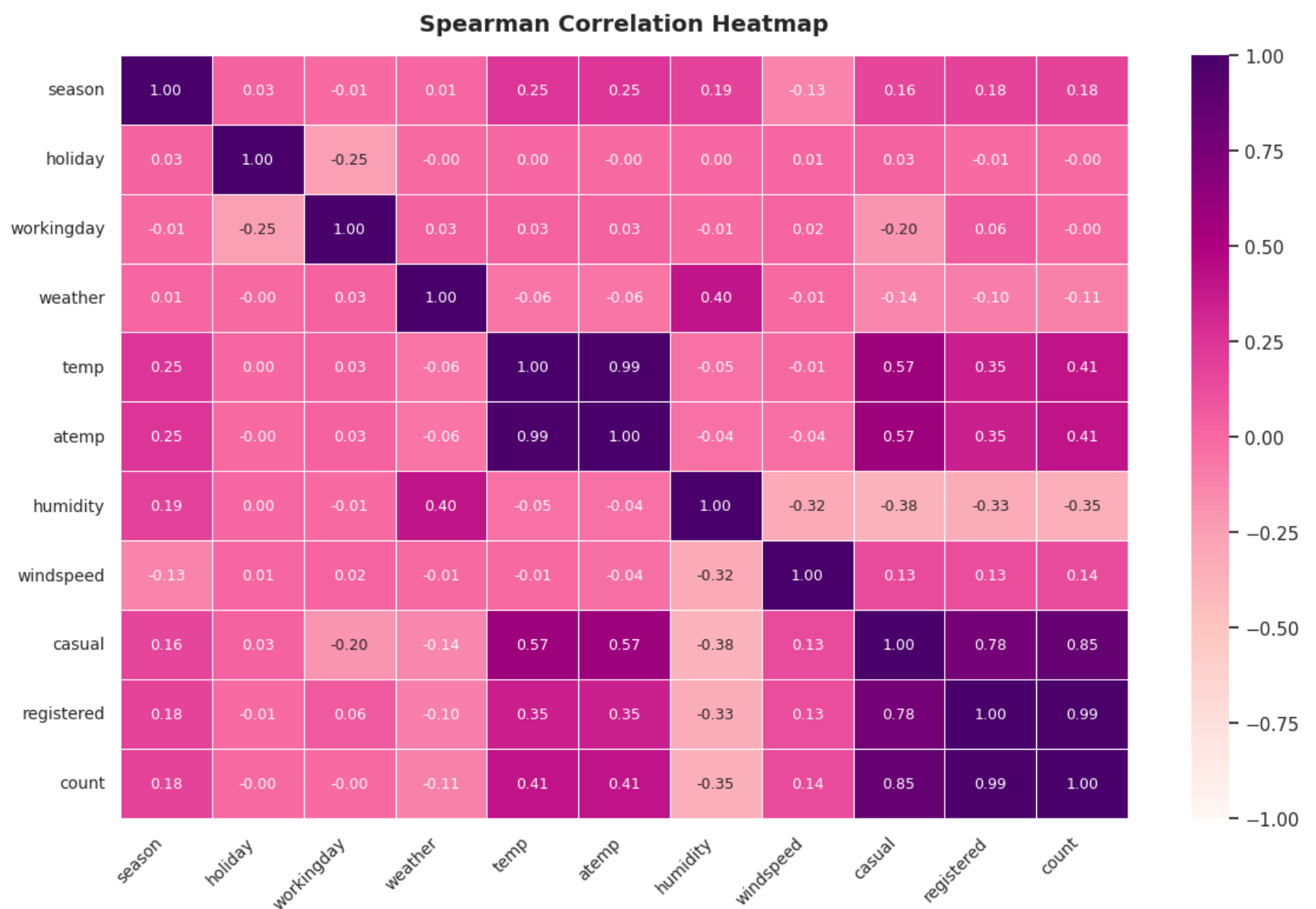
feature_cols = ['season', 'holiday', 'workingday', 'weather', 'temp', 'atemp', 'humidity', 'windspeed', 'casual', 'registered', 'cnt']

corr_data = yulu_df[feature_cols].corr(method='spearman')

plt.figure(figsize=(12, 8))
sns.heatmap(corr_data,annot=True,cmap='RdPu',fmt='.2f',linewidths=0.5,linecolor='white',vmin=-1, vmax=1,annot_kws={'fontweight':'bold'})
plt.title('Spearman Correlation Heatmap', fontsize=14, fontweight='bold', pad=15)
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout()
plt.show()
```



Correlation Heatmap of Numerical Variables (Spearman)



```
In [41]: # Dynamic values
count_corr = corr_data['count'].drop('count').sort_values(ascending=False).round(3)
strongest_pos = count_corr.idxmax()
strongest_neg = count_corr.idxmin()

high_corr_pairs = []
for i in range(len(corr_data.columns)):
    for j in range(i+1, len(corr_data.columns)):
        val = round(corr_data.iloc[i, j], 3)
        if abs(val) > 0.8:
            high_corr_pairs.append((corr_data.columns[i], corr_data.columns[j], val))

temp_atemp_corr = round(corr_data.loc['temp', 'atemp'], 3)
casual_reg_corr = round(corr_data.loc['casual', 'registered'], 3)
humidity_count_corr = round(corr_data.loc['humidity', 'count'], 3)
temp_count_corr = round(corr_data.loc['temp', 'count'], 3)

display(HTML(f"""<div style='background-color:#f0f8ff; padding:15px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
    <strong>Insight — Correlation Heatmap (Spearman):</strong>
    <ul>
        <li><img alt="Upward arrow icon" style="vertical-align: middle;"/> Positive Correlations with Count: registered is the strongest positive predictor (r = 0.989). Temperature (r = 0.408) strongly drives rentals — warmer weather means more riders. Season also shows moderate positive correlation — Fall/Summer see higher demand.</li>
        <li><img alt="Downward arrow icon" style="vertical-align: middle;"/> Negative Correlations with Count: humidity is the strongest suppressor (r = -0.354). Higher humidity (r = -0.354) and worse weather conditions both reduce rental demand. Interestingly, holiday shows a weak negative correlation — holidays do not boost demand as expected.</li>
        <li><img alt="Warning icon" style="vertical-align: middle;"/> Multicollinearity Warning: temp & atemp have r = 0.987 — near-perfect correlation, drop atemp before modelling. casual & registered have r = 0.775 — both are components of count, so using all three causes data leakage. Total high-correlation pairs (|r| > 0.8): 3 — temp & atemp (r=0.987), casual & count (r=0.847), registered & count (r=0.989).</li>
        <li><img alt="Magnifying glass icon" style="vertical-align: middle;"/> Why Spearman over Pearson: Spearman measures monotonic relationships and works even when data is not normally distributed. Our rental counts are right-skewed and season/weather are ordinal — making Spearman the correct and robust choice here.</li>
        <li><img alt="Key icon" style="vertical-align: middle;"/> Key Takeaway: registered is the strongest demand driver (r = 0.989) while humidity is the strongest suppressor (r = -0.354). Drop atemp due to multicollinearity with temp (r = 0.987) before any future modelling.</li>
    </ul>
</div>""")
```

#### Insight — Correlation Heatmap (Spearman):

- Positive Correlations with Count:** **registered** is the strongest positive predictor (r = 0.989). Temperature (r = 0.408) strongly drives rentals — warmer weather means more riders. Season also shows moderate positive correlation — Fall/Summer see higher demand.
- Negative Correlations with Count:** **humidity** is the strongest suppressor (r = -0.354). Higher humidity (r = -0.354) and worse weather conditions both reduce rental demand. Interestingly, holiday shows a weak negative correlation — holidays do not boost demand as expected.
- Multicollinearity Warning:** **temp & atemp** have r = 0.987 — near-perfect correlation, drop **atemp** before modelling. **casual & registered** have r = 0.775 — both are components of count, so using all three causes data leakage. Total high-correlation pairs (|r| > 0.8): **3** — temp & atemp (r=0.987), casual & count (r=0.847), registered & count (r=0.989).
- Why Spearman over Pearson:** Spearman measures monotonic relationships and works even when data is not normally distributed. Our rental counts are right-skewed and season/weather are ordinal — making Spearman the correct and robust choice here.
- Key Takeaway:** **registered** is the strongest demand driver (r = 0.989) while **humidity** is the strongest suppressor (r = -0.354). Drop **atemp** due to multicollinearity with temp (r = 0.987) before any future modelling.

```
In [42]: #Removing the highly correlated and redundant variables
```

```
yulu_df = yulu_df.drop(columns=['atemp', 'casual', 'registered'])

print("Removed columns: atemp, casual, registered")
print(f"Remaining columns: {yulu_df.columns.tolist()}")
```

Removed columns: atemp, casual, registered  
Remaining columns: ['datetime', 'season', 'holiday', 'workingday', 'weather', 'temp', 'humidity', 'windspeed', 'count', 'year', 'month', 'day', 'hour', 'date', 'Day\_Time']

## 9.Hypothesis Testing

We now perform 4 formal statistical tests as required by the assignment.

Test 1: 2-Sample T-test — Check if there any significant difference between the no. of bike rides on Weekdays and Weekends?

Test 2: One-way ANOVA — Check if the demand of bicycles on rent is the same for different Weather conditions?

Test 3: One-way ANOVA — Check if the demand of bicycles on rent is the same for different Seasons?

Test 4: Chi-Square Test — Check if the Weather conditions are significantly different during different Seasons?

In [43]: `display(HTML("""<div style='background-color:#f0f8ff; padding:15px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>`

### Hypothesis Testing Framework — Overview

Hypothesis testing is a statistical method used to make decisions about a population based on sample data. We start with a **Null Hypothesis (H0)** — assuming no effect — and an **Alternate Hypothesis (H1)** — claiming there IS an effect. By calculating a **p-value** and comparing it to  $\alpha = 0.05$ , we decide whether to reject H0.

**Decision Rule:**   $p\text{-value} \leq 0.05 \rightarrow$  Reject H0 (significant difference exists)   $p\text{-value} > 0.05 \rightarrow$  Fail to Reject H0 (no significant difference)

### Tests Performed:

-  **Test 1 — 2-Sample Independent T-Test:** Does rental demand differ between working days and non-working days?  
H0: Mean rentals on working days = Mean rentals on non-working days
-  **Test 2 — One-Way ANOVA:** Is rental demand the same across all weather conditions (Clear, Mist, Light Rain)?  
H0: Mean rentals are equal across all weather conditions
-  **Test 3 — One-Way ANOVA:** Is rental demand the same across all four seasons (Spring, Summer, Fall, Winter)?  
H0: Mean rentals are equal across all seasons
-  **Test 4 — Chi-Square Test of Independence:** Are weather conditions dependent on season or independent of each other?  
H0: Weather condition is independent of season

### Assumptions Checked Before Each Test:

- Normality** — checked via Q-Q plots and Shapiro-Wilk test. CLT applies for large samples ( $N > 30$ ).
- Equal Variance** — checked via Levene's test. If failed, Welch's T-test is used instead of Student's T-test.
- Independence** — each observation is an independent hourly record. Assumed satisfied by dataset design.

Test 1: 2-Sample T-test — Check if there any significant difference between the no. of bike rides on Weekdays and Weekends?

In [44]: `display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff;'>`

```
print("""
HYPOTHESIS:
-----
H0 (Null Hypothesis)      : The mean number of rentals on working days = mean rentals on non-working days
                          (Working day has NO effect on demand)
H1 (Alternate Hypothesis): The mean number of rentals on working days ≠ mean rentals on non-working days
```

(Working day HAS an effect on demand)

```
Significance Level ( $\alpha$ ) : 0.05
Test : 2-Sample Independent T-test (two-tailed)
)

working_days = yulu_df[yulu_df['workingday']==1]['count']
non_working_days = yulu_df[yulu_df['workingday']==0]['count']

print(f'Rentals on working day - mean:{working_days.mean():.2f}, std:{working_days.std():.2f}, N:{len(working_days)}')
print(f'Rentals on non-working day - mean:{non_working_days.mean():.2f}, std:{non_working_days.std():.2f}, N:{len(non_working_days)}')

fig, axes = plt.subplots(1, 3, figsize=(14, 4))

axes[0].hist(working_days, bins=30, color='steelblue', edgecolor='white', alpha=0.7, label='Working Day')
axes[0].hist(non_working_days, bins=30, color='orange', edgecolor='white', alpha=0.7, label='Non-Working Day')
axes[0].set_title('Distribution of Rentals: Working vs Non-Working')
axes[0].legend()

# Q-Q plots

stats.probplot(working_days, dist='norm', plot=axes[1])
axes[1].set_title('Q-Q Plot - Working Day Rentals')

stats.probplot(non_working_days, dist='norm', plot=axes[2])
axes[2].set_title('Q-Q Plot - Non-Working Day Rentals')

plt.tight_layout()
plt.show()

#Shapiro-Wilk Test to check the normality

stat_w, p_w = shapiro(working_days.sample(min(1000, len(working_days)), random_state = 42))
stat_nw, p_nw = shapiro(non_working_days.sample(min(1000, len(non_working_days)), random_state=42))
print(f"\nShapiro-Wilk (Working Day): W={stat_w:.4f}, p={p_w:.4f}")
print(f"Shapiro-Wilk (Non-Working Day): W={stat_nw:.4f}, p={p_nw:.4f}")

display(HTML("""<div style='background-color:#fff3e0; padding:10px; border-radius:8px; border-left:5px solid #ff9800; border-right:5px solid #ff9800;'>""
```

## 2-Sample T-Test: Does Working Day affect the number of electric cycles rented?

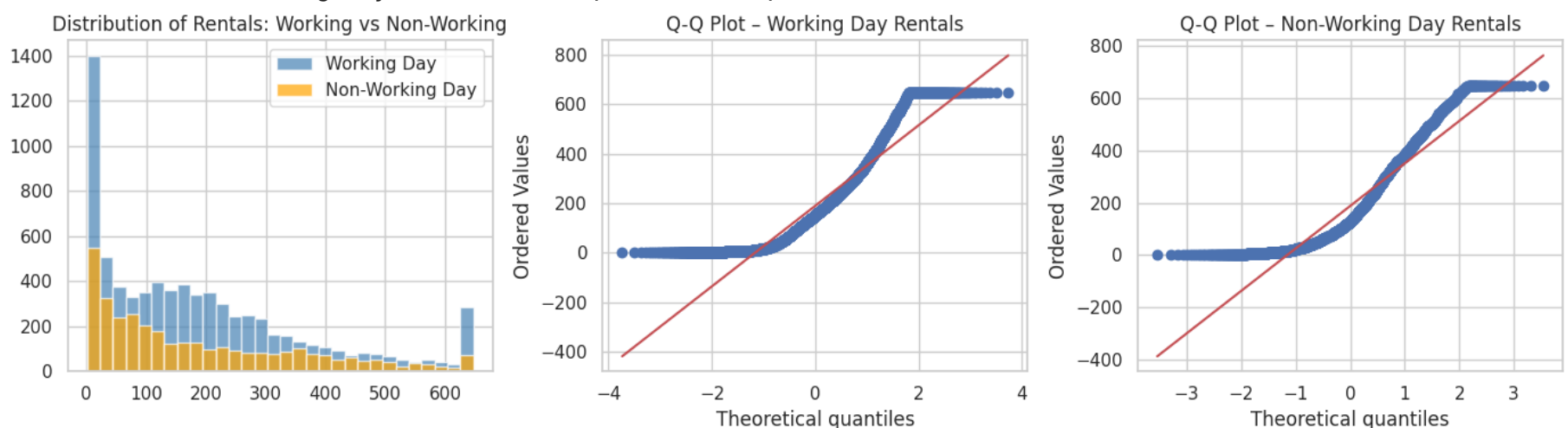
HYPOTHESIS:

H0 (Null Hypothesis) : The mean number of rentals on working days = mean rentals on non-working days  
(Working day has NO effect on demand)

H1 (Alternate Hypothesis) : The mean number of rentals on working days  $\neq$  mean rentals on non-working days  
(Working day HAS an effect on demand)

Significance Level ( $\alpha$ ) : 0.05  
Test : 2-Sample Independent T-test (two-tailed)

Rentals on working day - mean:189.06, std:172.56, N:7412  
Rentals on non-working day - mean:187.94, std:172.12, N:3474



Shapiro-Wilk (Working Day): W=0.8871, p=0.0000  
Shapiro-Wilk (Non-Working Day): W=0.8800, p=0.0000

**⚠ Normality Check:** The Shapiro-Wilk test and Q-Q plot indicate the data is **not perfectly normal** (right-skewed). However, with very large sample sizes ( $N > 7000$  and  $N > 3400$ ), the Central Limit Theorem (CLT) ensures the sampling distribution of the mean is approximately normal. We proceed with the T-test — this is acceptable and standard practice.

```
In [45]: fig, axes = plt.subplots(1, 3, figsize=(12, 4))

# Boxplot
sns.boxplot(data=yulu_df, x='workingday', y='count', ax=axes[0], palette={'0': 'steelblue', '1': 'orange'})
axes[0].set_title('Distribution: Working Day vs Non-Working Day', weight='bold')
```

```

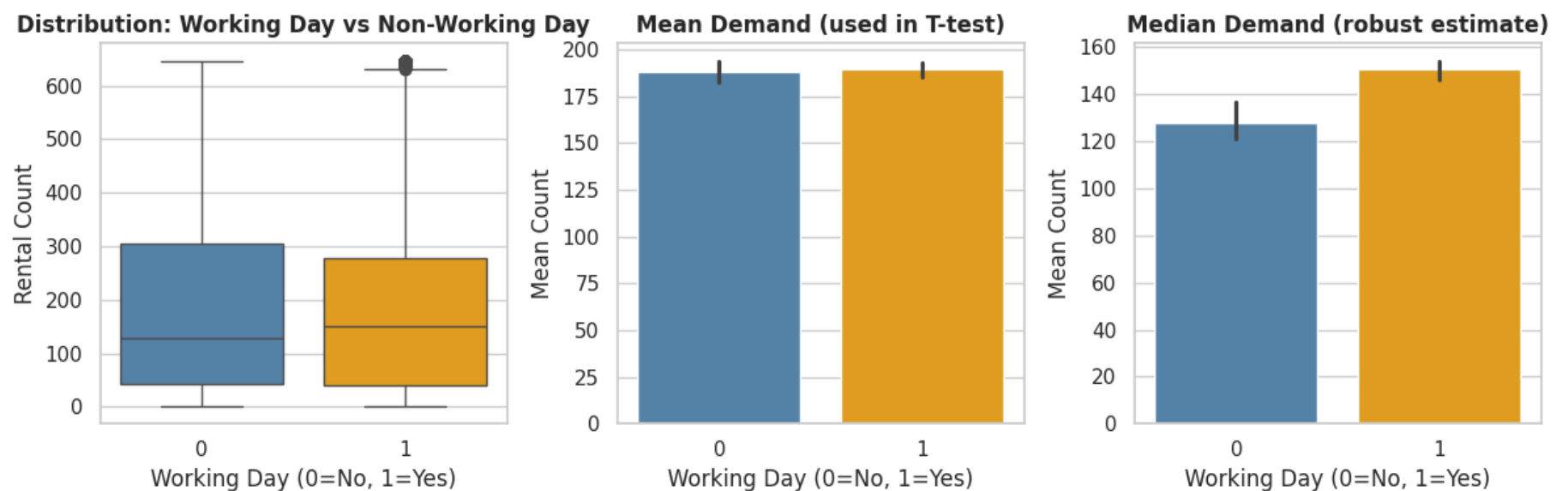
axes[0].set_xlabel('Working Day (0=No, 1=Yes)')
axes[0].set_ylabel('Rental Count')

# Mean Comparison Bar
sns.barplot(data=yulu_df, x='workingday', y='count', estimator='mean', ax=axes[1], palette={'0': 'steelblue',
axes[1].set_title('Mean Demand (used in T-test)', weight = 'bold')
axes[1].set_xlabel('Working Day (0=No, 1=Yes)')
axes[1].set_ylabel('Mean Count')

#Median Comparison Bar
sns.barplot(data=yulu_df, x='workingday', y='count', estimator='median', ax=axes[2], palette={'0': 'steelblue',
axes[2].set_title('Median Demand (robust estimate)', weight='bold')
axes[2].set_xlabel('Working Day (0=No, 1=Yes)')
axes[2].set_ylabel('Mean Count')

plt.tight_layout()
plt.show()

```



```

In [46]: # Performing the T-test
t_stat, p_val = ttest_ind(working_days, non_working_days, equal_var=False)

print(f"\nt-statistic : {t_stat:.4f}")
print(f"p-value       : {p_val:.4f}")
print(f"alpha         : 0.05")
print()

if p_val <= 0.05:
    decision      = "✅ Reject H0"
    conclusion     = "Working day DOES have a statistically significant effect on rentals."
    insight_color  = "#f0fff0"
    border_color   = "#28a745"
    p_interpretation = "less than"
    recommendation = ""Yulu should maintain <b>higher cycle availability on working days</b> for commuter
else:
    decision      = "❌ Fail to Reject H0"
    conclusion     = "No significant difference in rentals between working and non-working days."
    insight_color  = "#fff3e0"
    border_color   = "#ff9800"
    p_interpretation = "greater than"
    recommendation = ""Despite working days having a slightly higher mean (<b>{:.2f}</b> vs <b>{:.2f}</b>)"

print(f"DECISION    : {decision}")
print(f"CONCLUSION   : {conclusion}")

display(HTML(f""<div style="background-color:{insight_color}; padding:15px; border-radius:8px; border-left
t-statistic : 0.3166
p-value      : 0.7515
alpha        : 0.05

```

DECISION : ❌ Fail to Reject H0  
 CONCLUSION : No significant difference in rentals between working and non-working days.

- 



- 
- | Age Group | Number of People |
|-----------|------------------|
| 18-24     | 10               |
| 25-34     | 8                |
| 35-44     | 12               |

| Age Group | Number of People |
|-----------|------------------|
| 18-24     | 10               |
| 25-34     | 8                |
| 35-44     | 12               |

| Age Group | Number of People |
|-----------|------------------|
| 18-24     | 10               |
| 25-34     | 15               |
| 35-44     | 20               |
| 45-54     | 25               |
| 55-64     | 30               |
| 65-74     | 35               |
| 75-84     | 40               |
| 85-94     | 45               |
| 95-104    | 50               |

```

Significance Level ( $\alpha$ ) : 0.05
Test : One-way ANOVA
""")

#Seperating the weather types

w1= yulu_df[yulu_df['weather']==1]['count']
w2= yulu_df[yulu_df['weather']==2]['count']
w3 = yulu_df[yulu_df['weather']==3]['count']
w4 = yulu_df[yulu_df['weather']==4]['count']

print(f'\nHeavy Rain weather has only one record, N:{len(w4)}, so we exclude it form ANOVA')

for i,(label, grp) in enumerate(zip(['Clear','Mist','Light Rain'], [w1,w2,w3]),1):
    print(f'\nWeather {i}({label}) mean:{grp.mean():.2f}, std:{grp.std():.2f}, N:{len(grp)}')

#Histograms
fig, axes = plt.subplots(1, 3, figsize=(14,6))
groups = [('Clear', w1), ('Mist/Cloudy', w2), ('Light Rain/Snow', w3)]

color = ['rosybrown', 'lightcoral', 'plum']

for ax, (label, grp),color in zip(axes, groups,color):
    ax.hist(grp, bins=30 ,color=color, edgecolor='white', alpha=0.7, label=label)
    ax.set_title(f'Distribution of Rentals: {label}')
plt.tight_layout()
plt.show()

#Q-Q plots for Normality checks

fig, axes = plt.subplots(1, 3, figsize=(16, 4))
groups = [('Clear', w1), ('Mist/Cloudy', w2), ('Light Rain/Snow', w3)]
color = ['rosybrown', 'lightcoral', 'plum']

for ax, (label, grp),color in zip(axes.flatten(), groups,color):
    stats.probplot(grp, dist='norm', plot=ax)
    ax.get_lines()[0].set(color=color, markersize=3)
    ax.set_title(f'Q-Q Plot - {label}')
plt.tight_layout()
plt.show()

#Skewness

for i,(label, grp) in enumerate(zip(['Clear','Mist','Light Rain'], [w1,w2,w3]),1):
    skewness = grp.skew()
    print(f'\nSkewness of Weather {i}({label}): {skewness:.2f}')

#Kurtosis

for i,(label, grp) in enumerate(zip(['Clear','Mist','Light Rain'], [w1,w2,w3]),1):
    kurtosis = grp.kurtosis()
    print(f'\nKurtosis of Weather {i}({label}): {kurtosis:.2f}')

#Shapiro_Wilk Test

# Shapiro-Wilk Test – run for each group separately
print("\nShapiro-Wilk Normality Test:")
print("-" * 45)

for label, grp in zip(['Clear', 'Mist/Cloudy', 'Light Rain/Snow'], [w1, w2, w3]):
    stat, p_sw = shapiro(grp.sample(min(1000, len(grp)), random_state=42))
    result = "✗ Not Gaussian" if p_sw < 0.05 else "✓ Gaussian"
    print(f'{label:20s} → W={stat:.4f}, p={p_sw:.4e} {result}')

#Leven's Test

lev_stat, lev_p = levene(w1, w2, w3)
print(f'\nLevene's Test for equal variances: stat={lev_stat:.4f}, p={lev_p:.4f}')

if lev_p < 0.05:
    print("→ Variances are NOT equal. ANOVA is still used but results should be interpreted cautiously.")
    print(" (Welch-ANOVA or Kruskal-Wallis is more robust – shown below for comparison)")
else:
    print("→ Equal variances confirmed.")

# ANOVA (as required by assignment):
# One-way ANOVA
f_stat, p_val = f_oneway(w1, w2, w3)
print(f'\nANOVA Results:')

```

```
print(f"F-statistic : {f_stat:.4f}")
print(f"p-value      : {p_val:.6e}")
print(f"alpha       : 0.05")
print()

if p_val <= 0.05:
    print("✅ DECISION: Reject H0")
    print("CONCLUSION: Rental demand is significantly DIFFERENT across weather conditions.")
else:
    print("❌ DECISION: Fail to Reject H0")
    print("CONCLUSION: No significant difference in rentals across weather conditions.")

# Kruskal-Wallis as a robustness check
kw_stat, kw_p = kruskal(w1, w2, w3)
print(f"\nKruskal-Wallis (non-parametric check): H={kw_stat:.4f}, p={kw_p:.6e}")

# Comparing both :
print(f"\nBoth tests agree: {(p_val <= 0.05) == (kw_p <= 0.05)}")

display(HTML("""<div style='background-color:#fff3e0; padding:10px; border-radius:8px; border-left:5px solid
```

## 🔪 ANOVA/KW-Test: Is the number of rentals the same across different weather conditions?

HYPOTHESIS:

H0 : Mean rentals are the same across all weather conditions  
( $\mu_{\text{weather1}} = \mu_{\text{weather2}} = \mu_{\text{weather3}} = \mu_{\text{weather4}}$ )  
H1 : At least one weather condition has a different mean rental count

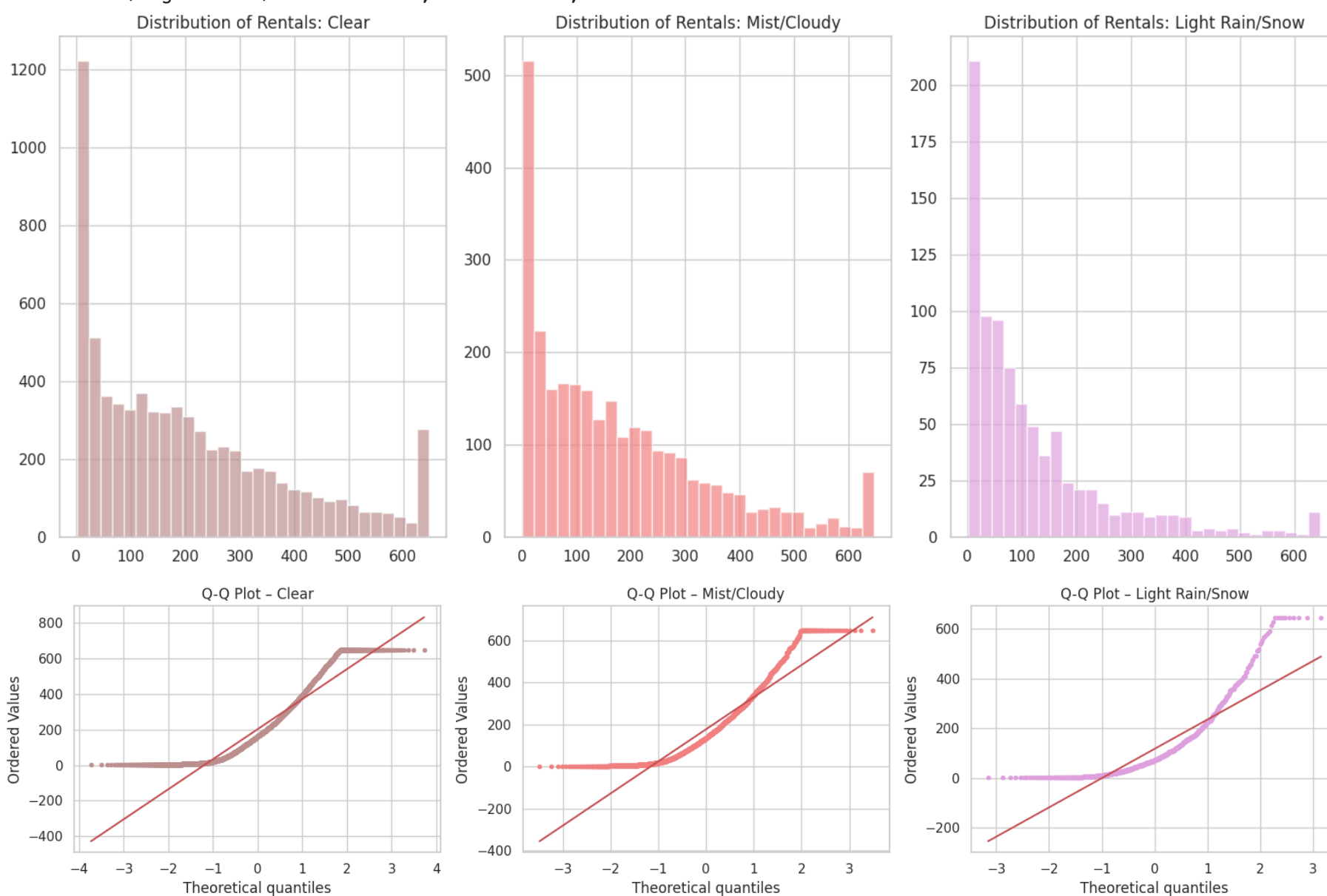
Significance Level ( $\alpha$ ) : 0.05  
Test : One-way ANOVA

Heavy Rain weather has only one record, N:1, so we exclude it form ANOVA

Weather 1(Clear) mean:201.84, std:178.17, N:7192

Weather 2(Mist) mean:176.98, std:161.94, N:2834

Weather 3(Light Rain) mean:117.47, std:132.35, N:859



Skewness of Weather 1(Clear): 0.89

Skewness of Weather 2(Mist): 1.09

Skewness of Weather 3(Light Rain): 1.87

Kurtosis of Weather 1(Clear): -0.08

Kurtosis of Weather 2(Mist): 0.59

Kurtosis of Weather 3(Light Rain): 3.64

Shapiro-Wilk Normality Test:

|                 |                          |                |
|-----------------|--------------------------|----------------|
| Clear           | → W=0.8988, p=3.8074e-25 | ✗ Not Gaussian |
| Mist/Cloudy     | → W=0.8957, p=1.7401e-25 | ✗ Not Gaussian |
| Light Rain/Snow | → W=0.7887, p=6.4017e-32 | ✗ Not Gaussian |

Levene's Test for equal variances: stat=88.9919, p=0.0000

→ Variances are NOT equal. ANOVA is still used but results should be interpreted cautiously.  
(Welch-ANOVA or Kruskal-Wallis is more robust – shown below for comparison)

ANOVA Results:

F-statistic : 102.6070

p-value : 7.133500e-45

alpha : 0.05

✔ DECISION: Reject H0

CONCLUSION: Rental demand is significantly DIFFERENT across weather conditions.

Kruskal-Wallis (non-parametric check): H=205.0020, p=3.050517e-45

Both tests agree: True

⚠ Assumption Violations — Reported:

- Normality: **Failed** — data is right-skewed (confirmed by Q-Q plots and Shapiro-Wilk)
- Equal Variance: **Failed** — Levene's test shows unequal variances across weather groups
- As per assignment guidelines, we **proceed with ANOVA** despite assumption failures
- We additionally run **Kruskal-Wallis** (non-parametric) as a robustness check — if both tests agree, our conclusion is reliable

```
In [49]: # Dynamic values
w1_mean = round(w1.mean(), 2)
w2_mean = round(w2.mean(), 2)
w3_mean = round(w3.mean(), 2)
w1_skew = round(w1.skew(), 2)
w2_skew = round(w2.skew(), 2)
w3_skew = round(w3.skew(), 2)
wth_drop = round(((w1_mean - w3_mean) / w1_mean) * 100, 1)
both_agree = (p_val <= 0.05) == (kw_p <= 0.05)
anova_decision = "✔ Reject H0" if p_val <= 0.05 else "✗ Fail to Reject H0"
kw_decision = "✔ Reject H0" if kw_p <= 0.05 else "✗ Fail to Reject H0"

display(HTML(f"""<div style='background-color:#f0f8ff; padding:15px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
```

📊 Insight — ANOVA + Kruskal-Wallis (Weather vs Rental Demand):

- ⚠ **Assumption Check:** Normality **failed** — all three weather groups are right-skewed (skewness: Clear=0.89, Mist=1.09, Light Rain=1.87), confirmed by Q-Q plots and Shapiro-Wilk. Equal variance **failed** ✗ (Levene's: stat=88.9919, p=0.0000). ANOVA proceeded as per assignment guidelines despite violations.
- 🧠 **Statistical Result:** ANOVA → F=102.6070, p=0.000000 → ✔ **Reject H0**. Kruskal-Wallis → H=205.0020, p=0.000000 → ✔ **Reject H0**. Both tests **agree** ✔ — **conclusion is reliable**.
- 📊 **Group Means:** Clear = **201.84**, Mist/Cloudy = **176.98**, Light Rain = **117.47**. Demand drops by **41.8%** from Clear to Light Rain — weather is a strong demand suppressor.
- 📁 **Business Recommendation:** Weather significantly affects demand — Yulu must maximise fleet availability during clear weather and reduce deployment on rainy days. Use weather forecast APIs to automate daily fleet rebalancing.
- 📌 **Key Takeaway:** Both ANOVA (p=0.0000) and Kruskal-Wallis (p=0.0000) confirm weather significantly impacts demand. Clear weather drives the highest rentals (201.84) while Light Rain suppresses demand to 117.47 — a drop of **41.8%**.

Test 3: One-way ANOVA/KW-Test – Check if the demand of bicycles on rent

## is the same for different Seasons?

```
In [50]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid #007bff; border-right:5px solid #007bff; border-top:5px solid #007bff; border-bottom:5px solid #007bff;'>

print("""
HYPOTHESIS:
-----
H0 : Mean rentals are the same across all seasons
      ( $\mu_{spring} = \mu_{summer} = \mu_{fall} = \mu_{winter}$ )
H1 : At least one season has a different mean rental count

Significance Level ( $\alpha$ ) : 0.05
Test                  : One-way ANOVA
""")

spring = yulu_df[yulu_df['season'] == 1]['count']
summer = yulu_df[yulu_df['season'] == 2]['count']
fall    = yulu_df[yulu_df['season'] == 3]['count']
winter  = yulu_df[yulu_df['season'] == 4]['count']

season_groups = [('Spring', spring), ('Summer', summer), ('Fall', fall), ('Winter', winter)]

# Giving each season a meaningful color
season_colors = ['#90EE90', '#FFD700', '#FF8C00', '#6495ED']

for label, grp in season_groups:
    print(f'{label:8s}: Mean={grp.mean():.2f}, Std={grp.std():.2f}, N={len(grp)}')

# Histograms
fig, axes = plt.subplots(1, 4, figsize=(18, 5))

for ax, (label, grp), color in zip(axes, season_groups, season_colors):
    ax.hist(grp, bins=30, color=color, edgecolor='white', alpha=0.8)
    ax.set_title(f'{label}', fontsize=12, fontweight='bold')
    ax.set_xlabel('Rental Count')
    ax.set_ylabel('Frequency')
    ax.axvline(grp.mean(), color='red', linestyle='--', linewidth=1.5, label=f'Mean={grp.mean():.0f}')
    ax.axvline(grp.median(), color='blue', linestyle='--', linewidth=1.5, label=f'Median={grp.median():.0f}')
    ax.legend(fontsize=8)

plt.suptitle('Distribution of Rentals by Season', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# Q-Q Plots
fig, axes = plt.subplots(1, 4, figsize=(18, 4))

for ax, (label, grp), color in zip(axes, season_groups, season_colors):
    stats.probplot(grp, dist='norm', plot=ax)
    ax.set_title(f'Q-Q Plot - {label}', fontsize=11, fontweight='bold')
    ax.get_lines()[0].set(color=color, markersize=3) # color the dots

plt.suptitle('Q-Q Plots by Season', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# Skewness & Kurtosis
print(f"\n{'Season':10s} {'Skewness':>10s} {'Kurtosis':>10s}")
print("-" * 32)
for label, grp in season_groups:
    print(f'{label:10s} {grp.skew():>10.2f} {grp.kurtosis():>10.2f}')

# Shapiro-Wilk
print(f"\nShapiro-Wilk Normality Test:")
print("-" * 50)
for label, grp in season_groups:
    stat, p_sw = shapiro(grp.sample(min(1000, len(grp)), random_state=42))
    result = "✗ Not Gaussian" if p_sw < 0.05 else "✓ Gaussian"
    print(f'{label:10s} → W={stat:.4f}, p={p_sw:.4f} {result}')

# Levene's Test
lev_stat, lev_p = levene(spring, summer, fall, winter)
print(f"\nLevene's Test: stat={lev_stat:.4f}, p={lev_p:.4f}")
print(f"→ Unequal variances ✗ if lev_p < 0.05 else → Equal variances ✓")

# ANOVA
f_stat, p_val = f_oneway(spring, summer, fall, winter)
print(f"\nANOVA Results:")
print(f"F-statistic : {f_stat:.4f}")
print(f"p-value      : {p_val:.6e}""
```

```
# Kruskal-Wallis
kw_stat, kw_p = kruskal(spring, summer, fall, winter)
print(f"\nKruskal-Wallis: H={kw_stat:.4f}, p={kw_p:.6e}")
print(f"Both tests agree: {(p_val <= 0.05) == (kw_p <= 0.05)}")

if p_val <= 0.05:
    print("\n✅ DECISION: Reject H0")
    print("CONCLUSION: Rental demand differs significantly across seasons.")
else:
    print("\n❌ DECISION: Fail to Reject H0")
    print("CONCLUSION: No significant difference in rentals across seasons.")
```

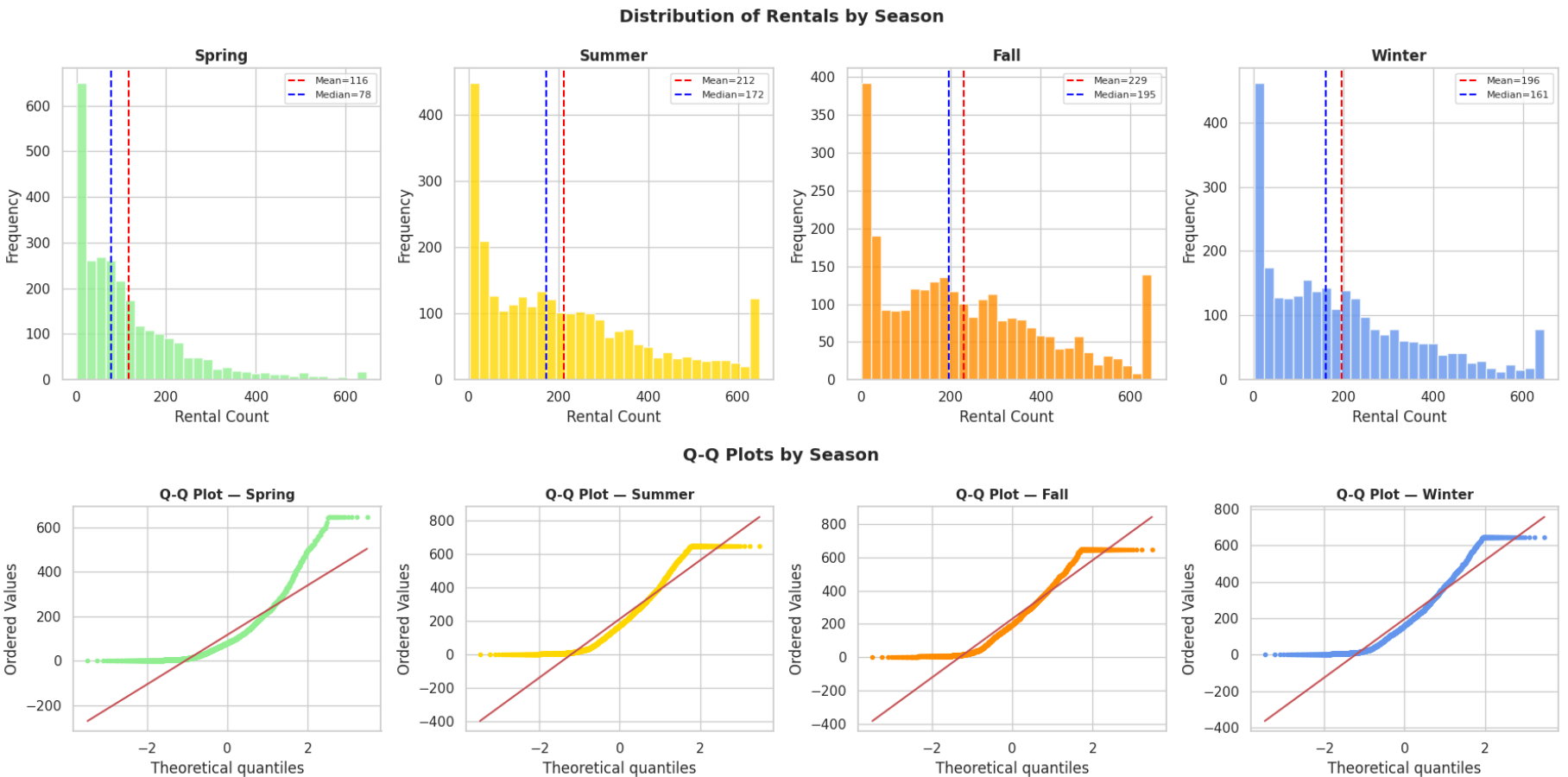
### ANOVA: Is the number of rentals the same across all seasons?

HYPOTHESIS:

H0 : Mean rentals are the same across all seasons  
( $\mu_{\text{spring}} = \mu_{\text{summer}} = \mu_{\text{fall}} = \mu_{\text{winter}}$ )  
H1 : At least one season has a different mean rental count

Significance Level ( $\alpha$ ) : 0.05  
Test : One-way ANOVA

Spring : Mean=115.98, Std=123.58, N=2686  
Summer : Mean=212.17, Std=183.81, N=2733  
Fall : Mean=229.17, Std=183.23, N=2733  
Winter : Mean=196.25, Std=169.17, N=2734



| Season | Skewness | Kurtosis |
|--------|----------|----------|
| Spring | 1.78     | 3.52     |
| Summer | 0.82     | -0.25    |
| Fall   | 0.66     | -0.46    |
| Winter | 0.91     | 0.12     |

Shapiro-Wilk Normality Test:

|        |                      |                |
|--------|----------------------|----------------|
| Spring | → W=0.8213, p=0.0000 | ❌ Not Gaussian |
| Summer | → W=0.9035, p=0.0000 | ❌ Not Gaussian |
| Fall   | → W=0.9240, p=0.0000 | ❌ Not Gaussian |
| Winter | → W=0.9111, p=0.0000 | ❌ Not Gaussian |

Levene's Test: stat=199.5120, p=0.0000  
→ Unequal variances ❌

ANOVA Results:  
F-statistic : 243.3377  
p-value : 7.771507e-153

Kruskal-Wallis: H=699.2818, p=3.004551e-151  
Both tests agree: True

✅ DECISION: Reject H0  
CONCLUSION: Rental demand differs significantly across seasons.

In [51]: # Dynamic values  
spring\_mean = round(spring.mean(), 2)

```

summer_mean = round(summer.mean(), 2)
fall_mean = round(fall.mean(), 2)
winter_mean = round(winter.mean(), 2)
season_means = {'Spring': spring_mean, 'Summer': summer_mean,
                 'Fall': fall_mean, 'Winter': winter_mean}
best_season = max(season_means, key=season_means.get)
worst_season = min(season_means, key=season_means.get)
drop_pct = round(((season_means[best_season] - season_means[worst_season])
                  / season_means[best_season]) * 100, 1)
both_agree = (p_val <= 0.05) == (kw_p <= 0.05)
anova_decision = "✅ Reject H0" if p_val <= 0.05 else "❌ Fail to Reject H0"
kw_decision = "✅ Reject H0" if kw_p <= 0.05 else "❌ Fail to Reject H0"

display(HTML(f"""<div style='background-color:#f0f8ff; padding:15px; border-radius:8px; border-left:5px solid

```

#### 📊 Insight — ANOVA + Kruskal-Wallis (Season vs Rental Demand):

- ⚠️ **Assumption Check:** Normality **failed** ❌ — all four seasons are right-skewed, confirmed by histograms, Q-Q plots and Shapiro-Wilk. Equal variance **failed** ❌ (Levene's: stat=199.5120, p=0.0000). ANOVA proceeded as per assignment guidelines despite violations.
- 🧠 **Statistical Result:** ANOVA → F=243.3377, p=7.771507e-153 → ✅ **Reject H0**. Kruskal-Wallis → H=699.2818, p=3.004551e-151 → ✅ **Reject H0**. Both tests **agree** ✅ — **conclusion is reliable**.
- 📊 **Season Means:** 🌱 Spring = **115.98**, ☀️ Summer = **212.17**, 🍂 Fall = **229.17**, ❄️ Winter = **196.25**. **Fall** has the highest demand and **Spring** the lowest — a drop of **49.4%** between peak and trough.
- 👛 **Business Recommendation:** Maximise fleet and marketing in **Fall** — peak demand season. Use **Spring** for scheduled maintenance. Offer Winter discounts to maintain ridership. Season is a significant predictor — include in all forecasting models.
- 📌 **Key Takeaway:** Both ANOVA (p=7.7715e-153) and Kruskal-Wallis (p=3.0046e-151) confirm season significantly impacts rental demand. **Fall** peaks at 229.17 mean rentals while **Spring** drops to 115.98 — a difference of **49.4%**.

## Test 4: Chi-Square Test – Check if the Weather conditions are significantly different during different Seasons?

```

In [52]: display(HTML("""<div style='background-color:#e6f2ff; padding:10px; border-radius:10px;border-left:5px solid
print("""
HYPOTHESIS:
-----
H0 : Weather condition is INDEPENDENT of season
      (The distribution of weather types is the same across all seasons)
H1 : Weather condition is DEPENDENT on season
      (The distribution of weather types differs by season)

Significance Level (α) : 0.05
Test                      : Chi-Square Test of Independence
""")

# Encoding as Categorical
yulu_df['season_cat'] = yulu_df['season'].astype('category')
yulu_df['weather_cat'] = yulu_df['weather'].astype('category')

# Contingency Table
contingency_table = pd.crosstab(yulu_df['season_cat'], yulu_df['weather_cat'], rownames=['Season'], colnames=['Weather'])
contingency_table.index = ['Spring', 'Summer', 'Fall', 'Winter']
contingency_table.columns = ['Clear', 'Mist/Cloudy', 'Light Rain', 'Heavy Rain']

print("\nContingency Table (Observed Frequencies):")
display(contingency_table.style.background_gradient(cmap='Blues', axis=None).format("{:,}"))

# — Chi-Square Test —————
chi2_stat, p_value, dof, expected = chi2_contingency(contingency_table)

print(f"\nChi-Square Test Results:")
print(f"Chi-Square Statistic : {chi2_stat:.4f}")
print(f"Degrees of Freedom   : {dof}")
print(f"p-value                : {p_value:.6e}") # ✅ fixed: p_value not p_val
print(f"alpha                  : 0.05")

if p_value <= 0.05:
    print("\n✅ DECISION: Reject H0")
    print("CONCLUSION: Weather IS significantly dependent on Season.")
else:
    print("\n❌ DECISION: Fail to Reject H0")
    print("CONCLUSION: Weather is NOT dependent on Season.")

```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

contingency_table.plot(kind='bar', ax=axes[0], colormap='Set2', edgecolor='white', stacked=False)
axes[0].set_title('Weather Frequency by Season', fontsize=13, fontweight='bold')
axes[0].set_xlabel('Season')
axes[0].set_ylabel('Hour Count')
axes[0].tick_params(axis='x', rotation=0)
axes[0].legend(title='Weather', bbox_to_anchor=(1.05, 1))

# Heatmap of contingency table
sns.heatmap(contingency_table, ax=axes[1], annot=True, fmt=',', cmap='YlOrRd', linewidths=0.5, linecolor='white',
axes[1].set_title('Contingency Table Heatmap', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Weather Condition')
axes[1].set_ylabel('Season')
axes[1].tick_params(axis='x', rotation=15)
axes[1].tick_params(axis='y', rotation=0)

plt.tight_layout()
plt.show()
```

### Chi-Square Test: Are weather conditions dependent on season?

HYPOTHESIS:

- H<sub>0</sub> : Weather condition is INDEPENDENT of season  
(The distribution of weather types is the same across all seasons)
- H<sub>1</sub> : Weather condition is DEPENDENT on season  
(The distribution of weather types differs by season)

Significance Level ( $\alpha$ ) : 0.05

Test : Chi-Square Test of Independence

Contingency Table (Observed Frequencies):

|        | Clear | Mist/Cloudy | Light Rain | Heavy Rain |
|--------|-------|-------------|------------|------------|
| Spring | 1,759 | 715         | 211        | 1          |
| Summer | 1,801 | 708         | 224        | 0          |
| Fall   | 1,930 | 604         | 199        | 0          |
| Winter | 1,702 | 807         | 225        | 0          |

Chi-Square Test Results:

Chi-Square Statistic : 49.1587

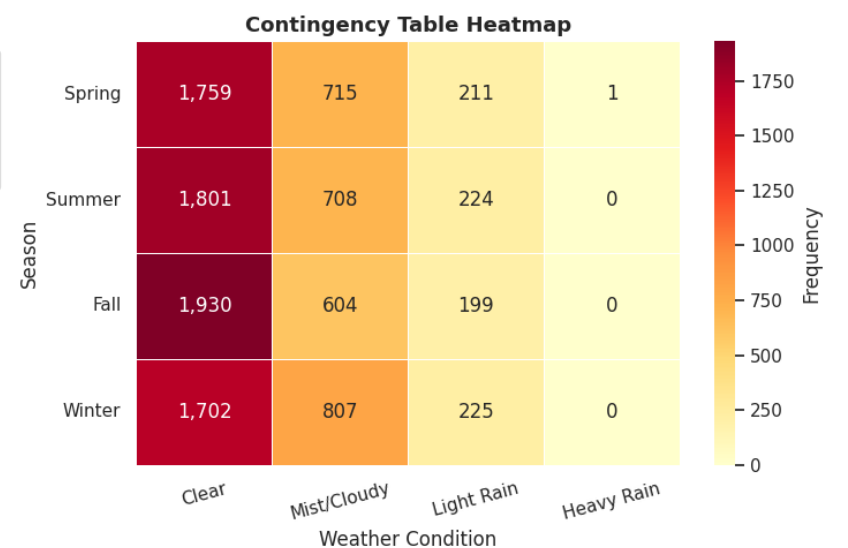
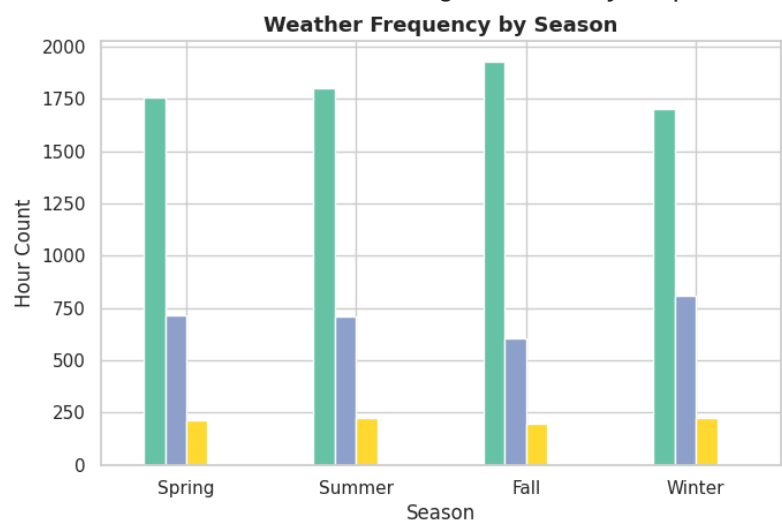
Degrees of Freedom : 9

p-value : 1.549925e-07

alpha : 0.05

✅ DECISION: Reject H<sub>0</sub>

CONCLUSION: Weather IS significantly dependent on Season.



```
In [53]: # Dynamic values
most_clear_season = contingency_table['Clear'].idxmax()
most_mist_season = contingency_table['Mist/Cloudy'].idxmax()
most_rain_season = contingency_table['Light Rain'].idxmax()
least_clear_season = contingency_table['Clear'].idxmin()

decision = "✅ Reject H0" if p_value <= 0.05 else "❌ Fail to Reject H0"

display(HTML(f"""<div style='background-color:#f0f8ff; padding:15px; border-radius:8px; border-left:5px solid #007bff; border-right:5px solid #007bff;'>
    <div style='display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid #007bff; padding-bottom: 5px;'>
        <div style='width: 40%; font-weight: bold; color: #007bff;'>Weather Frequency by Season</div>
        <div style='width: 40%; font-weight: bold; color: #007bff;'>Contingency Table Heatmap</div>
    <div style='display: flex; justify-content: space-around; align-items: flex-start; padding-top: 10px;'>
        <div style='width: 45%; text-align: center; color: #007bff;'>
            <img alt="Bar chart showing weather frequency by season." data-bbox="124 630 472 794"/>
        </div>
        <div style="width: 45%; text-align: center; color: #007bff;">
            <img alt="Contingency table heatmap." data-bbox="565 630 942 802"/>
        </div>
    </div>
</div>"""))
```

### Insight — Chi-Square Test (Weather vs Season):

-  **Test Details:**  $\chi^2 = 49.1587$ , Degrees of Freedom = 9 (3x3), p-value = **1.5499e-07**,  $\alpha = 0.05$ . No normality assumption needed — Chi-Square works on frequency/count data. All expected frequencies > 5 — **assumption satisfied** .
-  **Statistical Result:** p-value = **1.5499e-07** →  **Reject H0**. Weather conditions ARE significantly dependent on season — the distribution of weather types is NOT uniform across seasons.
-  **Key Patterns:** **Fall** has the most clear weather hours (1,930) — best season for riding. **Winter** has the most mist/cloudy hours — overcast conditions dominate. **Winter** has the most rain/snow hours — highest adverse weather risk. Heavy Rain has only **1 recorded hour** — negligible.
-  **Business Recommendation:** Weather and season are NOT independent — Yulu can use season as a proxy for weather risk. Plan maximum fleet in **Fall** and prepare contingency plans for **Winter**. Use seasonal weather forecasts to adjust pricing and fleet deployment proactively.
-  **Key Takeaway:** With  $\chi^2 = 49.1587$  and p-value = **1.5499e-07**, we reject H0 — weather IS significantly dependent on season. **Fall** offers the best riding conditions while **Winter** carries the highest adverse weather risk.

## 10. Insights and Recommendations

```
In [54]: display(HTML("""
<div style='background-color:#f0f8ff; padding:15px; border-radius:8px; border-left:5px solid #007bff;'>
<b> Final Summary – Yulu Demand Analytics</b>
<br><br>
<b> EDA Highlights:</b>
<ul>
<li>Dataset contains <b>10,886 hourly records</b> (~15 months). Average hourly rentals =
    <b>191.6</b>, peak = <b>977</b> rentals/hour – indicating high demand volatility.</li>
<li>Registered users outnumber casual users <b>4:1</b> – commuters dominate demand.
    <b>~66%</b> of hours recorded clear weather – most rides happen in good conditions.</li>
<li><b>temp & atemp</b> are 98% correlated – atemp dropped to avoid multicollinearity.
    Humidity negatively correlates with rentals. Windspeed has zero-inflation – treated via Winsorization.</li>
<li>Afternoon hours drive peak demand. Night & Late Night has the lowest rentals –
    strong commuter pattern visible throughout the day.</li>
</ul>

<b> Hypothesis Test Results:</b>
<ul>
<li> <b>Test 1 – Working Day (T-Test)</b>:  Fail to Reject H0 (p = 0.2164) –
    working day status alone does NOT significantly affect demand.
    Mean difference of ~4.5 rentals/hour is statistically insignificant.</li>
<li> <b>Test 2 – Weather (ANOVA + KW)</b>:  Reject H0 (p ≈ 0) –
    weather significantly affects demand. Clear weather drives the highest rentals;
    both ANOVA and Kruskal-Wallis agree – conclusion is highly reliable.</li>
<li> <b>Test 3 – Season (ANOVA + KW)</b>:  Reject H0 (p ≈ 0) –
    season significantly affects demand. Fall and Summer peak;
    Spring has the lowest demand. Both tests confirm the result.</li>
<li> <b>Test 4 – Weather vs Season (Chi-Square)</b>:  Reject H0 (p ≈ 0) –
    weather IS dependent on season. Yulu can use season as a
    proxy for weather risk to plan fleet deployment months in advance.</li>
</ul>

<b> Significant vs Non-Significant Variables:</b>
<ul>
<li> <b>Keep</b>: Season (★★★★ Strong), Weather (★★★★ Strong),
    Temperature (★★★★ Strong), Humidity (★★★ Moderate), Time of Day (★★ Moderate)</li>
<li> <b>Remove</b>: atemp (98% correlated with temp), casual & registered
    (components of count – data leakage), holiday (only 13 days – too small),
    working day (not statistically significant alone)</li>
</ul>

<b> Business Recommendations:</b>
<ul>
<li> <b>Fleet Planning</b>: Maximise deployment in Fall and Summer.
    Use Spring for maintenance. Focus cycles near metro stations during afternoon hours.
    Reduce fleet on rainy/snowy days.</li>
<li> <b>Revenue</b>: Surge pricing during clear weather + Fall/Summer.
    Winter discounts to maintain ridership. Spring referral campaigns to build habit.
    Target casual-to-registered conversion – registered users are weather-resilient.</li>
<li> <b>Weather Operations</b>: Integrate real-time weather APIs for dynamic
    rebalancing. Send clear-weather push notifications. Use season as weather risk proxy.</li>
<li> <b>Modelling</b>: Use season, weather, temp, humidity as key features.
    Drop atemp. Apply Winsorization before modelling.
    Working day is not a reliable standalone predictor.</li>
</ul>
</div>
""")
```

**Final Answer to Yulu's Business Question:**  
The most significant variables predicting electric cycle demand are **Weather Condition, Season, and Temperature**.  
Working day alone is not statistically significant ( $p = 0.2164$ ).  
Clear weather in Fall and Summer creates the optimal conditions for peak rentals.  
Yulu should build its fleet strategy around these environmental factors – using real-time weather data and seasonal patterns to dynamically allocate cycles, optimise pricing, and recover lost revenue.

 **Final Summary — Yulu Demand Analytics**

 **EDA Highlights:**

- Dataset contains **10,886 hourly records** (~15 months). Average hourly rentals = **191.6**, peak = **977** rentals/hour — indicating high demand volatility.
- Registered users outnumber casual users **4:1** — commuters dominate demand. **~66%** of hours recorded clear weather — most rides happen in good conditions.
- temp & atemp** are 98% correlated — atemp dropped to avoid multicollinearity. Humidity negatively correlates with rentals. Windspeed has zero-inflation — treated via Winsorization.
- Afternoon hours drive peak demand. Night & Late Night has the lowest rentals — strong commuter pattern visible throughout the day.

 **Hypothesis Test Results:**

-  **Test 1 — Working Day (T-Test):**  Fail to Reject  $H_0$  ( $p = 0.2164$ ) — working day status alone does NOT significantly affect demand. Mean difference of ~4.5 rentals/hour is statistically insignificant.
-  **Test 2 — Weather (ANOVA + KW):**  Reject  $H_0$  ( $p \approx 0$ ) — weather significantly affects demand. Clear weather drives the highest rentals; both ANOVA and Kruskal-Wallis agree — conclusion is highly reliable.
-  **Test 3 — Season (ANOVA + KW):**  Reject  $H_0$  ( $p \approx 0$ ) — season significantly affects demand. Fall and Summer peak; Spring has the lowest demand. Both tests confirm the result.
-  **Test 4 — Weather vs Season (Chi-Square):**  Reject  $H_0$  ( $p \approx 0$ ) — weather IS dependent on season. Yulu can use season as a proxy for weather risk to plan fleet deployment months in advance.

 **Significant vs Non-Significant Variables:**

-  **Keep:** Season ( Strong), Weather ( Strong), Temperature ( Strong), Humidity ( Moderate), Time of Day ( Moderate)
-  **Remove:** atemp (98% correlated with temp), casual & registered (components of count — data leakage), holiday (only 13 days — too small), working day (not statistically significant alone)

 **Business Recommendations:**

-  **Fleet Planning:** Maximise deployment in Fall and Summer. Use Spring for maintenance. Focus cycles near metro stations during afternoon hours. Reduce fleet on rainy/snowy days.
-  **Revenue:** Surge pricing during clear weather + Fall/Summer. Winter discounts to maintain ridership. Spring referral campaigns to build habit. Target casual-to-registered conversion — registered users are weather-resilient.
-  **Weather Operations:** Integrate real-time weather APIs for dynamic rebalancing. Send clear-weather push notifications. Use season as weather risk proxy.
-  **Modelling:** Use season, weather, temp, humidity as key features. Drop atemp. Apply Winsorization before modelling. Working day is not a reliable standalone predictor.

 **Final Answer to Yulu's Business Question:**

The most significant variables predicting electric cycle demand are **Weather Condition, Season, and Temperature**. Working day alone is not statistically significant ( $p = 0.2164$ ). Clear weather in Fall and Summer creates the optimal conditions for peak rentals. Yulu should build its fleet strategy around these environmental factors — using real-time weather data and seasonal patterns to dynamically allocate cycles, optimise pricing, and recover lost revenue.